

**SAPIENTIA ERDÉLYI MAGYAR TUDOMÁNYEGYETEM
CSÍKSZEREDAI KAR
GAZDASÁGI INFORMATIKA SZAK**

D I P L O M A D O L G O Z A T

Végzős hallgató:

Károlyi Krisztián

Témavezető:

Dr. Madaras Szilárd, egyetemi adjunktus

2024

**SAPIENTIA ERDÉLYI MAGYAR TUDOMÁNYEGYETEM
CSÍKSZEREDAI KAR
GAZDASÁGI INFORMATIKA SZAK**

D I P L O M A D O L G O Z A T

**A MUNKANÉLKÜLISÉG
ELŐREJELZÉSE
PYTHONBEN, ARIMA ÉS
MLP MODELLEKKEL**

Végzős hallgató:

Károlyi Krisztián

Témavezető:

Dr. Madaras Szilárd, egyetemi adjunktus

2024

Kivonat

A dolgozat fő célja az, hogy összehasonlítsa a hagyományos Box-Jenkins módszeren alapuló ARIMA modellek és a többretegű perceptron (MLP) mesterséges neurális hálóból (ANN) készített regressziós modellek előrejelzési teljesítményét és pontosságát, a székelyföldi megyék munkanélküliségi rátáit modellezve. A legjobban általánosító ARIMA és MLP modellekkel előrejelzést készítettem hat hónapra, a 2024 március és 2024 augusztus közötti időszakra.

Tanító adatbázisnak Hargita, Kovászna és Maros megye 2010 január és 2023 február közötti havi munkanélküliségi rátáit, tesztelő adatbázisnak pedig a 2023 március és 2024 február közötti megfigyeléseket használtam fel. Utóbbival a modellek általánosító képességét vizsgáltam az MSE, RRMSE és MAPE mutatók figyelembevételével.

Minden számítást Python-ben végeztem. A kutatás során egy Python programot készítettem, amelyhez egy Django webalkalmazás segítségével biztosítok felhasználói felületet. Az alkalmazás Excel fájlban előkészített idősorokat tud feldolgozni. Összekapcsoltam a különböző Python csomagokat, amelyek az idősorok elemzéséhez, a különböző prediktív modellek készítéséhez, és az adatok ábrázolásához szükségesek. Ezeket egy felhasználóbarát, egységes webes felületbe szerveztem. A program az eredményeket grafikonokkal vizualizálja és táblázatokkal foglalja össze, ezzel megkönnyítve a modellek megfelelő szerkezetének és paramétereinek megtalálását. Előnye, hogy a felhasználó számára nehezen elérhető Python modulokat egy interaktív felhasználóbarát webfelületen elérhetővé teszi.

Az eredményekből azt a következtetést vontam le, hogy mindhárom idősor esetében az MLP modellek jobban illeszkedtek a tanító adatbázisra, és jobban általánosítottak, mint az ARIMA modellek.

Az előrejelzések szerint egy enyhe tavaszi emelkedést követően, a nyár végére Kovászna megyében 4.4%, Hargita megyében 4%, Maros megyében pedig 3% körül fog alakulni a munkanélküliségi ráta.

Rezumat

Principalul obiectiv al lucrării este de a compara performanța și acuratețea prognozei modelelor ARIMA bazate pe metoda tradițională Box-Jenkins și pe modelul de regresie cu rețea neuronală artificială (RNA) cu perceptron multistrat (MLP), modelând ratele șomajului în județele secuiesti. Cele mai bune modele generalizatoare ARIMA și MLP și modelele au fost utilizate pentru a produce o prognoză pe șase luni pentru perioada martie 2024 - august 2024.

Ca set de date de instruire am folosit ratele lunare ale șomajului pentru județele Harghita, Covasna și Mureș între ianuarie 2010 și februarie 2023, iar ca set de date de testare am folosit observații între martie 2023 și februarie 2024. Acesta din urmă a fost utilizat pentru a testa capacitatea de generalizare a modelelor prin luarea în considerare a indicatorilor MSE, RRMSE și MAPE.

Toate calculele au fost efectuate în Python. Am creat un program Python pentru care am oferit o interfață de utilizator folosind o aplicație web Django. Aplicația poate procesa serii temporale pregătite într-un fișier Excel.

Am conectat diferitele pachete Python necesare pentru a analiza seriile temporale, pentru a construi diferite modele predictive și pentru a reprezenta grafic datele. Le-am organizat într-o interfață web unificată și ușor de utilizat. Programul vizualizează rezultatele cu ajutorul unor grafice și le rezumă în tabele, facilitând astfel găsirea structurii și a parametrilor potriviți pentru modele. Avantajul este că pune la dispoziție module Python care sunt dificil de accesat de către utilizator pe o interfață web interactivă ușor de utilizat.

Din rezultatele obținute, am ajuns la concluzia că, pentru toate cele trei serii de timp, modelele MLP se potrivesc mai bine bazei de date de formare și sunt mai generalizate decât modelele ARIMA.

După o ușoară creștere în primăvară, rata șomajului este prognozată să se situeze în jurul valorii de 4,4% în județul Covasna, 4% în județul Harghita și 3% în județul Mureș până la sfârșitul verii.

Abstract

The main objective of the paper is to compare the forecasting performance and accuracy of ARIMA models based on the traditional Box-Jenkins method and the multi-layer perceptron (MLP) artificial neural network (ANN) regression model, modelling unemployment rates in the counties of Seco. The best generalizing ARIMA and MLP models and models were used to produce a six-month forecast from March 2024 to August 2024. As a training dataset, we used monthly unemployment rates for Harghita, Covasna, and Mures counties between January 2010 and February 2023, and as a test dataset, we used observations between March 2023 and February 2024. The latter was used to test the generalizability of the models by considering the MSE, RRMSE, and MAPE indicators.

All calculations were performed in Python. We created a Python program to provide a user interface using a Django web application. The application can process time series prepared in an Excel file. We connected the various Python packages needed to analyse the time series, build multiple predictive models, and plot the data. We organized them into a unified and easy-to-use web interface. The program visualizes the results with graphs and summarizes them in tables, making it easy to find the right structure and parameters for the models. Its advantage is that it makes available Python modules that are difficult for the user to access on an interactive user-friendly web interface.

From the results, we concluded that in the case of the all-time series, MLP models fit the training database better and generalize better than ARIMA models. After a slight increase in spring, the unemployment rate is forecast to be around 4.4% in Covasna County, 4% in Harghita County, and 3% in Mures County by the end of summer.

Tartalomjegyzék

1. Bevezetés.....	1
2. Szakirodalmi áttekintés	2
3. A kutatás módszertana és elméleti megalapozás	5
3.1. A munkanélküliségi ráta meghatározása	5
3.2. Előrejelzések pontosságát mérő mutatók.....	6
3.3. Box-Jenkins eljárás.....	7
3.3.1. A stacionaritás vizsgálata.....	9
3.3.2. Autoregresszív integrált mozgóátlag modellek (ARIMA)	9
3.4. Neurális hálózatok.....	11
3.4.1. Neuron, Perceptron, MLP	12
3.4.2. Az MLP modell optimális szerkezetének kiválasztása.....	13
3.4.3. ReLU aktivációs függvény.....	14
3.4.4. Ellenőrzött tanítási stratégia	15
3.4.5 Optimalizációs algoritmusok.....	17
4. A kutatási gyakorlati része	18
4.1. A Django webalkalmazás működése	18
4.1.1 Adatbeolvasás.....	19
4.1.2. A beolvasott idősorok tárolása munkamenet segítségével.....	20
4.2. Az idősorok elemzése	21
4.2.1. Az adatok eloszlása.....	23
4.2.2. A stacionaritás vizsgálata.....	24
4.3. Az ARIMA modellek implementációja	26
4.3.1 A modellek illesztése	27
4.3.2 A modellek tesztelése	29
4.3.3. Előrejelzések az ARIMA modellekkel	30
4.4. A neuronhálók implementációja	32
4.4.1. A tanító és tesztelő adatbázis előkészítése	32
4.4.2. Az adatok előfeldolgozása	33
4.4.3. Az MLP modellek implementációja.....	35
4.4.4. A modellek tanítása és tesztelése	36
4.4.5. Rekurzív előrejelzés az MLP modellekkel	38
5. Eredmények	39

5. Következtetések	40
6. Irodalomjegyzék	41
7. Mellékletek	44

1. Bevezetés

A munkanélküliség hosszú ideje jelentős gazdasági mutató és központi téma a közgazdaságtani kutatásokban. A munkanélküliség alakulása és változása az adott régió gazdasági egészségét tükrözi, és fontos információkat szolgáltat a gazdasági kilátásokról. A COVID-19 pandémia súlyos gazdasági hatásokkal járt számos országban, beleértve Romániát is.

A járvány okozta korlátozások és gazdasági visszaesés jelentős mértékben befolyásolta a munkaerőpiacot és az álláskeresési helyzetet. Kis- és nagyvállalkozások egyaránt bezártak, bocsátottak el dolgozókat vagy küldték őket kényszerszabadságra. A munkanélküliségi ráta havi idősorainak elemzése és a megbízható előrejelzések készítése kulcsfontosságú eszköz lehet az adott régió gazdaságának és munkanélküliségi helyzetének változásainak megértésében, valamint az ehhez igazított gazdasági intézkedések megalapozásában (Davidescu és társai, 2021).

Ebben a dolgozatban Hargita, Kovászna és Maros megye 2010 január - 2022 július közötti havi munkanélküliségi rátáit vizsgáltam. Az adatokat romániai Nemzeti Statisztikai Hivatal (románul „Institutul Național de Statistică”) weboldaláról (TEMPO Online) töltöttem le.

A célom az volt, hogy statisztikai elemzést készítssek ezekről az idősorokról, valamint megvizsgáljam, hogy ezen idősorok esetében, további fél egy távlatában (2023 március – 2024 február) a Box-Jenkins eljárással készült ARIMA (autoregresszív integrált mozgóátlag), vagy a gépi tanulással működő MLP (multilayer perceptron) mesterséges neuronháló modellek nyújtanak pontosabb előrejelzéseket.

A legjobban általánosító modellekkel pedig további fél évre (2024 március – 2024 augusztus) készítettem előrejelzéseket a székelyföldi megyék munkanélküliségi rátáinak alakulására.

A számításokat Python-ben végeztem, amelyhez egy Django webalkalmazás segítségével biztosítok felhasználói felületet. Az volt a célom, hogy a nehezen elérhető Python modulokat felhasználóbarát módon elérhetővé tegyem és egy egységes felületbe szervezzem. A főbb funkciók a következők: statisztikai elemzés, ARIMA modell kiválasztásához szükséges tesztek (Box-Jenkins eljárás), az MLP regressziós modellek egyszerű készítése, ezenkívül adatvizualizáció táblázatok és grafikonok segítségével. A webalkalmazás bármilyen idősort képes feldolgozni, így számos további, hasonló tematikájú kutatáshoz hasznos lehet, illetve egy felhasználó könnyen készíthet előrejelzéseket a saját idősorára.

2. Szakirodalmi áttekintés

A COVID-19 világjárványt követő gazdasági körülmények kapcsán kiemelt szerepet kapott a munkanélküliség vizsgálata, ezen belül a különböző munkanélküliségi előrejelző módszertanok. A szakirodalomból ismert konvencionális előrejelzési modellek a Box-Jenkins módszertanon alapulnak. A munkanélküliségi ráták előrejelzésére sok tudományos cikkben használták a Box-Jenkins módszertant.

Madaras (2014) Románia országos havi munkanélküliségi rátáit vizsgálta 2005 január – 2013 június között és 1 éves távlatban készített előrejelzést ARIMA (1, 1, 4) modellel.

Adenomon (2017) Nigéria munkanélküliségi rátáit jelezte előre 4 év távlatában, ARIMA (2, 1, 2) modellel, 1972 és 2014 közötti éves adatok felhasználásával.

Tóth és társai (2021) a Visegrádi országok munkanélküliségének alakulását modellezték és jelezték előre ARIMA modellekkel, 2010 és 2021 közötti havi adatokkal. Az ARIMA modelleket az AIC mutató alapján választották ki.

Ramli és társai (2018) Malajziában, 1960 és 2016 közötti adatok segítségével jelezték előre a munkanélküliségi rátákat 10 éves távlatban, ARIMA (2, 1, 2) modellel. MAE, RMSE, MAPE hiba becslésekkel hasonlították össze az ARIMA és Holt-Winters modellek általánosító képességét.

Didiharyono és Muhammad (2020) Indiában jelezték előre egy évre a regionális munkanélküliségi rátákat, Dél-Celebesz tartományban, ARIMA (1, 2, 1) modellel, 1968 és 2018 közötti negyedéves munkanélküliségi ráták felhasználásával.

A Box-Jenkins eljárással való előrejelzésre a különböző mesterséges neuronháló (ANN) modellek reális alternatívát jelentenek, számos kutatás foglalkozott már azzal, hogy a két megközelítést összehasonlítsák különböző idősorok esetében, a legtöbb alkalommal a gépi tanulás (*Machine Learning*) jobban teljesített. A modellek általánosító képességét, vagyis azt, hogy a tanító adatbázisra illesztett modell milyen pontossággal tudja becsülni a tanító adatbázison kívüli adatokat (teszt adatbázist), általában a következő mérőszámokkal hasonlítják össze: átlagos abszolút hiba (MAE), átlagos abszolút százalékos hiba (MAPE), az átlagos négyzetes hiba (MSE), ennek négyzetgyöke (RMSE), illetve az RRMSE, amely az RMSE-t normalizálja a valódi adatok átlagával.

Davidescu és társai (2021) eredményei alapján az NNAR (Neural Network Autoregression) típusú neuronháló modell jobban jelezte előre a romániai munkanélküliségi rátákat 2018 és 2020 között, mint a SARIMA, amely a szezonalitást is képes kezelni. A modellek teljesítményét a MAE, MAPE és RMSE mutatókkal hasonlították össze.

Ajoodha és Mulaudzi (2020) Dél-Afrika éves munkanélküliségi rátáinak előrejelzéséhez készítettek összehasonlító elemzést, ahol háromféle hagyományos – köztük az ARIMA – és hétféle gépi tanulást használó modell – köztük az MLP és LSTM – általánosító képességét vizsgálták, a MAPE és R^2 mérőszámok alapján. A legjobbnak az MLP, majd az ARIMA, végül az LSTM mutatkozott.

Madaras (2018) Hargita és Brassó megye esetében megállapította, hogy rövidtávon a mesterséges neuronháló-alapú NAR (nemlineáris autoregresszív) modell, középtávon a Box-Jenkins eljárással készített ARMA (1, 1) modellek nyújtottak pontosabb becseleket.

Tufaner és Sözen (2021) Törökországban végeztek hasonló kutatást, MSE hiba becslés alapján hasonlították össze az ARIMA (3, 1, 2) modell előrejelző pontosságát egy két rejtett réteggel rendelkező, logisztikus (szigmoid) aktivációs függvényt használó MLP neuronhálós modellel. Az MLP modell sokkal jobban teljesített.

D. S. Yamacli és S. Yamacli (2023) Szintén Törökországban kutatták ezt a területet, főleg arra voltak kíváncsiak, hogy hogyan teljesítenek az ARIMA és az ANN modellek a COVID-19 világjárvány által érintett időszak alatti megfigyelések felhasználása mellett és anélkül. Az derült ki az RMSE hiba becslés alapján, hogy az ARMA (2, 1) modell a járvány időszakán kívül jobban teljesített, azonban a bizonytalan gazdasági időszakban a két rejtett rétegű neuronhálós modell jobban teljesített.

Madaras (2024) öt Közép-európai ország munkanélküliségi rátáit modellezte és jelezte előre ARIMA, MLP, CNN, és random forest modellekkel. Eredményei alapján az MLP algoritmus teljesített a legjobban Csehország Magyarország, Lengyelország és Románia adataival, míg Szlovákia esetében a random forest algoritmus bizonyult a legjobbnak. Összességében az MLP és random forest modellek jobbnak bizonyultak, mint az ARIMA, és a CNN teljesített a leggyengébben.

A munkanélküliségi rátákon kívül számos tudományos cikkben vizsgálták az MLP és egyéb neurális hálózatok előrejelzési teljesítményét a gazdasági és pénzügyi idősorok terén. Az eredmények erősen függenek a vizsgált idősoroktól, azonban az MLP modellek sokszor jobbnak bizonyultak.

Derbentsev és társai (2020) négyféle ANN modell, köztük az MLP előrejelző képességét vizsgálták a Bitcoin, Ethereum, és az EUR-USD árfolyamokon. Az MLP és SGBM modellek 0.46-3.71%-os MAPE hibamutatóval jeleztek előre a teszt időszakra, így megállapították, hogy a gépi tanúlással való megközelítés valóban alkalmas a pénzügyi előrejelzések készítésére.

Sasu (2013) összehasonlította az MLP és az ARIMA előrejelző-képességét, tízféle idősort felhasználva, például foglalkoztatottsági adatokat, munkanélküliségi ráta, és banki letétek összegét. Eredményeik szerint, a MAPE mutató alapján az MLP modellek jobban teljesítettek.

Oukhouya és Himdi (2023) a marokkói tőzsdeindex (MSI 20) napi árának előrejelzésén tesztelték az XGBoost, SVR, LSTM, MLP neuronhálós modelleket. Megállapították, hogy a rácskereséses optimalizációs algoritmus (Grid Search) mellett az MLP és az SVG teljesítettek a legjobban. Az MAE, MSE, MAPE, RMSE, és R^2 mutatókat egyaránt figyelembe vették.

A gépi tanulásban (machine learning) többféle optimalizációs algoritmust és aktivációs függvényt használnak. A legelterjedtebb algoritmusok közé tartozik például az SGD (sztochasztikus gradiens süllyedés), ADAM (Adaptive Moment Estimation), L-BFGS (Limited-memory Broyden–Fletcher–Goldfarb–Shanno). Aktivációs függvényeknél elterjedtek például a logisztikus (szigmoid), ReLU, hiperbolikus tangens, lépcsőugrás függvények. Pardede és társai (2022) COVID-19 betegek osztályozásán vizsgálták az MLP modellek teljesítményét, különböző aktivációs függvények és optimalizációs algoritmusok mellett. Eredményeik szerint az L-BFGS algoritmus, ReLU és logisztikus aktivációs függvényekkel jobban teljesített, mint az ADAM vagy az SGD.

3. A kutatás módszertana és elméleti megalapozás

Ebben a fejezetben a kutatás során felhasznált elméleti részt mutatom be, amelyek fontosak a kutatás gyakorlati részének megértéséhez.

3.1. A munkanélküliségi ráta meghatározása

A dolgozat során felhasznált adatok Románia Nemzeti Statisztikai Hivatalától (románul „Institutul Național de Statistică”) Tempo Online adatbázisából származnak. A munkanélküliek a BIM (Biroul Internațional al Muncii) meghatározása szerint azok a 15-74 év közötti személyek, akik egyidejűleg teljesítik a következő három feltételt:

- a mérés pillanatában nincs bejelentett munkahelyük
- a következő két héten belül munkába tudnának állni
- az elmúlt négy hétben aktívan munkát kerestek.

A munkanélküliségi ráta egy százalékos arányszám: a munkanélküliek arányát mutatja a munkaerőhöz viszonyítva egy adott térségre. A gazdaságilag aktív népesség a bázisidőszakban az áruk és szolgáltatások előállítására rendelkezésre álló munkaerőt biztosító valamennyi személyt magában foglalja, beleértve a foglalkoztatottakat és a munkanélkülieket is (INSSE, 2016).

Tehát a munkanélküliségi ráta megmutatja, hogy a munkaképes lakosság hány százaléka nem rendelkezik a mérés pillanatában munkahellyel, azonban képes lenne és szeretne dolgozni. Ezt a statisztikai mutatót havonta és éves időszakokban is mérik, én a havi adatokkal dolgoztam, mivel rövidtávon könnyebb előrejelezni.

Azért modelleztem mindhárom székelyföldi megye munkanélküliségét, hogy átfogóbb képet kaphassak az ARIMA és MLP regressziós modellek teljesítményéről, mivel ezek a folyamatok alapvetően hasonló viselkedésűek, viszont nem biztos, hogy ugyanaz a módszertan jobban teljesít mindhárom idősor esetében.

3.2. Előrejelzések pontosságát mérő mutatók

Az előrejelzések pontosságának megállapításához a szakirodalomból ismert MSE, MAPE és RRMSE mutatókat használtam, amelyek minél alacsonyabbak, a modellnek annál pontosabbak az előrejelzései. Azért használják egyszerre regressziós modelleknél, mert mindegyik különböző aspektusokat emel ki a modell teljesítményének értékelésében, és így átfogóbb képet adnak a modell pontosságáról és megbízhatóságáról.

Mivel a MAPE-t a MAE képletéből, és az RRMSE az RMSE képletéből le lehet vezetni, ezért ezek képletét is leírtam, Madaras (2024) alapján.

Jelöljük n -nel a megfigyelések számát, t -vel az időpontot, az x_t -vel a megfigyelést (valódi értéket), és f_t -vel az előrejelzett értéket a t -edik időpontban.

Az átlagos abszolút hiba (MAE) a valódi adat és a becsült érték közötti eltérések abszolút értékeinek az átlaga. Az abszolút érték miatt nem veszi figyelembe a hiba irányát. Nem bünteti túlzottan a nagyobb hibákat, így érzékenyebb a kisebb hibákra. Nem ad információt a hiba irányáról (túl- vagy alábecslés).

$$MAE = \frac{1}{n} \sum_{t=1}^n |f_t - x_t| \quad (3.1)$$

Az átlagos abszolút százalékos eltérés (MAPE) százalékban kifejezve mutatja meg – tehát skálafüggetlen és könnyen értelmezhető – hogy mennyire nagy az átlagos abszolút eltérés a megfigyelés és a becsült értékek között. Nem bünteti különösen a nagyobb hibákat, mivel a hibák abszolút értékeivel dolgozik. Hasznos, ha az eltérések aránya fontosabb, mint az abszolút eltérések.

$$MAPE = \frac{1}{n} \sum_{t=1}^n \left| \frac{f_t - x_t}{x_t} \right| \quad (3.2)$$

Az átlagos négyzetes hiba (MSE) a tényleges és becsült adatok közötti különbségek négyzeteinek az átlaga. A hibák négyzetre emelése miatt érzékeny a kiugró értékekre, mivel nagyobb súlyt ad a nagyobb eltéréseknek, így jobban kiemeli a jelentős hibákat.

$$MSE = \frac{1}{n} \sum_{t=1}^n (x_t - f_t)^2 \quad (3.3)$$

Az átlagos négyzetes hiba gyöke (RMSE) az MSE négyzetgyöke, ezáltal az eredmény ugyanabban a mértékegységben van, mint az eredeti adatok. A megfigyelések és a modell által becsült értékek közötti négyzetes eltérések átlagos négyzetgyökét mutatja.

$$RMSE = \sqrt{\frac{1}{n} \sum_{t=1}^n (x_t - f_t)^2} \quad (3.4)$$

A relatív átlagos négyzetes eltérések gyöke (RRMSE) a becslési hibák szórása. Az RRMSE az RMSE-t normalizálja a megfigyelések átlagával. Az RRMSE az eredeti értékekhez viszonyítja a hibákat, ami fontos, ha az adatok skálája változik vagy az eredmények relatív jellegűek. Kevésbé érzékeny az adatok skálájának változásaira vagy a kiugró értékekre.

$$RRMSE = \sqrt{\frac{1}{n} \sum_{t=1}^n \left(\frac{x_t - f_t}{x_t} \right)^2} \quad (3.5)$$

A determinancia-együttható (R^2) azt mutatja, hogy a magyarázó változó milyen mértékben magyarázza a célváltozó változékonyságát. Az R^2 0 és 1 közötti értékeket vehet fel. Egy 1-hez közeli R^2 érték azt jelenti, hogy a modell jól illeszkedik az adatokhoz, vagyis a modell jól magyarázza az adatsor változásait. Azonban nem biztosít elég információt a modell által becsült értékek pontosságáról, ezért önmagában nem elég a modellek általánosító képességének vizsgálatához. (Sándor 2019)

$$R^2 = 1 - \frac{\sum e_i^2}{\sum (y_i - \bar{y})^2} \quad (3.6)$$

3.3. Box-Jenkins eljárás

A módszertan nevét két fő megalkotójáról, George Box-ról és Gwilym Jenkins-ről kapta. A fő lépései a következők, amelyeket a 3.1. ábrán szemléltetnek. Ez egy iteratív folyamat, amelyben az elemző folyamatosan finomítja és teszteli a modellt, amíg el nem éri a legjobb előrejelzési teljesítményt.

1) Identifikáció:

- a. Az idősor ábrázolása. Az esetleges szezonálisok, trendek itt is feltűnhetnek.
- b. A stacionaritás vizsgálata (például ADF és KPSS tesztek segítségével). Ha nem stacionárius az idősor, differenciálni kell (az I komponens rendjét növeljük).

- c. Az autoregresszív és a mozgóátlag komponens rendjeinek kezdeti behatárolása az auto- és parciális autokorrelációs függvények vizsgálatával.

2) Paraméterek becslése

- a. A modell illesztése a tanító idősorra, a komponensek együttthatóinak becslése. Általában valamilyen szoftverrel végzik, mivel rengeteg számítást igényel. A Python statmodels.ARIMA osztálya kiszámítja az értékeket.
- b. Az illesztés során kiszámított AIC és BIC mutatókat figyelembe vesszük. Minél kisebbek ezek a mutatók, a modell annál valószínűbb, hogy a legjobban teljesít.

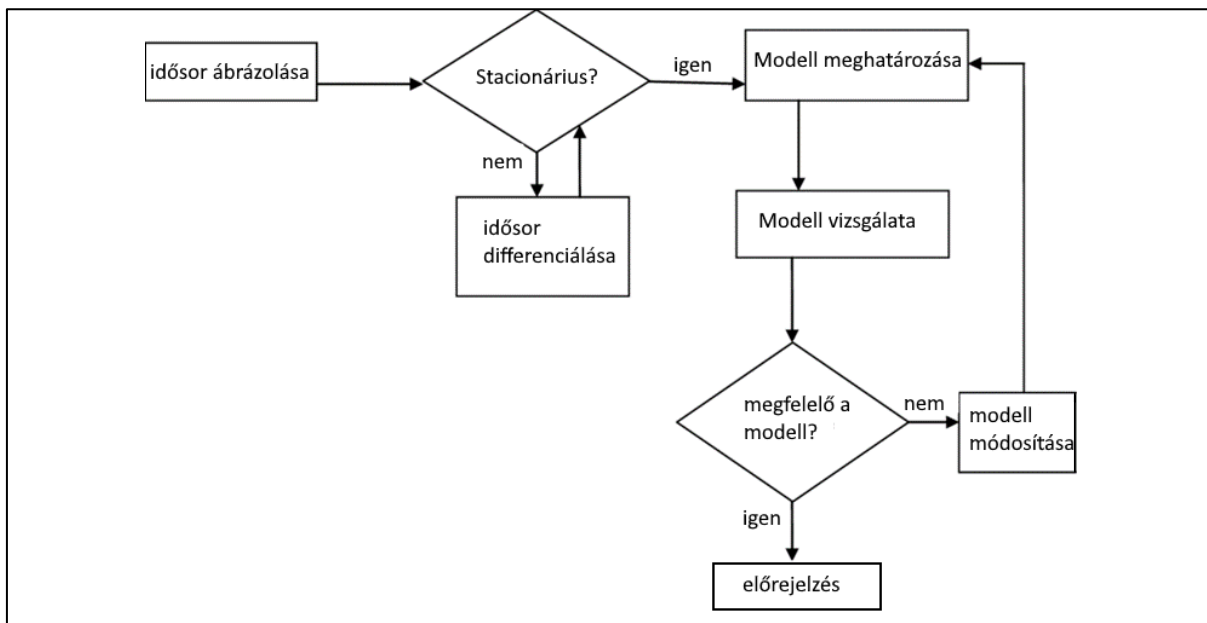
3) Modell diagnosztika:

- a. A reziduumok (tanító idősorra adott becslési hibák) eloszlásának, autokorrelációinak, varianciájának vizsgálata. A cél az, hogy a hibák fehérzajként viselkedjenek.

4) Előrejelzés.

- a. A modell által még nem ismert megfigyelések becslése, és azok pontosságának vizsgálata különböző hibamutatókkal (pl. MSE, RRMSE, MAPE).
- b. Előrejelzések készítése.

Ha modell becsléseinek reziduumai nem felelnek meg a modell diagnosztika során, vagy túl nagy hibával becsül a teszt adatokra, módosítani kell a modellt a modell komponenseinek rendjeit. (Sándor, 2019).



3.1. ábra. A Box-Jenkins eljárás folyamatábrája.

Forrás: saját szerkesztés.

A Box-Jenkins eljárás során a megfelelő AR/MA/ARMA/ARIMA modelleket az AIC mutató alapján választják ki véglegesen. Az AIC a becült modell illeszkedésének minőségét és a modell bonyolultságát összehasonlító mutató. Tehát az a modell illeszkedik a legjobban az eredeti értékekhez, amelynek az AIC kritérium értéke a legkisebb. Képlete:

$$AIC = 2k - 2 \ln(L), \quad (3.7)$$

ahol k a modell paramétereinek száma, L a modell valószínűségi értéke, ami azt méri, hogy mennyire valószínű, az, hogy a modell előállítja a megfelelő megfigyelt adatokat a paramétereinek ismeretében.

3.3.1. A stacionaritás vizsgálata

A stacionaritás az idősorok statisztikai tulajdonságainak időbeni (közelítő) állandóságát jelenti. Egy stacionárius idősor várható értéke, varianciája és autokorrelációs függvénye állandó, vagy csak időben állandó konstans eltolódásokkal változik. A stacionárius idősorok könnyebben modellezhetők és előrejelezhetőek (Sándor, 2019). A stacionaritást az ADF és KPSS egységgyök tesztekkel vizsgáltam Madaras és Lehel (2016) alapján.

Az Augmented Dickey-Fuller (ADF) teszt nullhipotézise (H_0) az, hogy az idősorban van egységgyökér, vagyis az idősor nem stacionárius míg az alternatív hipotézis (H_1) az, hogy nincs egységgyökér, tehát az idősor stacionáriusnak mondható. Ha a p -érték (szignifikancia szint) kisebb, mint 0.05, akkor elutasítjuk a nullhipotézist, tehát az idősor stacionárius, mert nincs kimutatható egységgyökér.

A Kwiatkowski-Phillips-Schmidt-Shin (KPSS) teszt nullhipotézise az, hogy az idősor szigorúan stacionárius (az ADF nullhipotézisével ellentétben), tehát nincs egységgyökér. Ha a p -érték nagyobb, mint 0.05, nem utasítjuk el a nullhipotézist, vagyis az idősor stacionárius.

3.3.2. Autoregresszív integrált mozgóátlag modellek (ARIMA)

Ebben a fejezetben az AR/MA/ARMA/ARIMA modelleket és komponenseit mutatom be Sándor (2019) alapján.

Az AR(p) tulajdonképpen egy többváltozós lineáris modell, ahol a független változók a függőváltozó késleltetett értékei. Az autoregressziós komponens (AR) azt jelenti, hogy az aktuális időpontbeli becült értéket p darab korábbi időpontbeli érték határozza meg. Egy p -rendű autoregresszív AR(p) folyamat, a (3.8) egyenlettel írható le:

$$y_t = \alpha + \phi_1 y_{t-1} \pm \dots + \phi_p y_{t-p} + \varepsilon_t, t = p + 1, \dots, n \quad (3.8)$$

Az $\phi_1, \dots, \phi_p$ az autoregresszív együtthatók, ezeket egy szoftver számolja ki általában. az ε_t a hibaváltozó, amit fehérzajnak feltételezünk, vagyis olyan folyamatnak, amelynek várható értéke 0, varianciája konstans és autokorrelációja 0, valamint a hibaváltozó késleltetett értékei között nincsen autokorreláció.

A mozgóátlag (MA) azt jelenti, hogy az aktuális időpontbeli értéket a korábbi időpontbeli hibák lineáris kombinációjaként becsüljük meg. Arra utal, hogy az aktuális érték korrelál az előző időpontbeli becslési hibákkal, és az q paraméter megadja a mozgóátlag rendjét, azaz hány korábbi hibaértéket használunk az aktuális érték becsléséhez. Egy q rendű mozgóátlag folyamat a (3.9) egyenlettel írható le:

$$y_t = \alpha + \theta_1 \varepsilon_t + \varepsilon_{t-1} + \dots + \theta_q \varepsilon_{t-q}, t = q + 1, \dots, n \quad (3.9)$$

$\theta_1 \dots \theta_q$ a becslési hibák súlyai, amelyeket programmal számolunk ki. A két folyamat kombinációja az ARMA (p, q) (autoregresszív mozgóátlag) folyamat, amely komplexebb idősorokat is képes leírni.

$$y_t = \alpha + \phi_1 y_{t-1} + \dots + \phi_p y_{t-p} + \varepsilon_t + \theta_1 \varepsilon_{t-1} + \dots + \theta_q \varepsilon_{t-q} \quad (3.10)$$

P az autoregressziós folyamat rendje, q a mozgóátlag folyamat rendje, y_t az idősorozat aktuális értéke, α a konstans érték, $\phi_1, \dots, \phi_p$ az autoregresszív együtthatók, ε_t az adott időpontbeli fehérzaj, $\theta_1, \dots, \theta_q$ a mozgóátlag együtthatók.

Az ARIMA (p, d, q) modellekben az I (integrated) azt jelenti, hogy az idősor differenciáljuk. A differenciálás célja az, hogy az idősorok stacionáriusok legyenek, ehhez az szükséges, hogy a szezonális mintázatokat eltávolítsuk. Ilyenkor az idősor különbségeit számoljuk ki az eggyel korábbi megfigyelés alapján:

$$\Delta Y_t = Y_t - Y_{t-1} \quad (3.11)$$

A mozgóátlag folyamat (MA) q rendjét elsősorban az idősor autokorrelációs függvényének ábrázolásával határozzuk meg. Egy idősor autokorrelációs függvénye (ACF) az autokorrelációk sorozata. Ha például első 2 késleltetett érték után hirtelen „bezuhanak” a korrelációs értékek, akkor $q = 2$, viszont, ha fokozatos a csökkenés, akkor $q = 0$.

Az autoregresszív (AR) p rendjét hasonlóan határozzuk meg, viszont itt a parciális autokorrelációs függvényt kell ábrázolni. A PACF azt mutatja meg, hogy egy adott időpontbeli

megfigyelés és az előző időpontok közötti autokorreláció mennyire marad meg, ha kizárjuk az idősor többi részéből származó közvetlen korrelációt. (Sándor 2019)

Ezek leginkább irányadó módszerek, nem biztos, hogy az így kapott modellek lesznek a legjobbak. Érdekes lefuttatni többféle modell illesztését, és a legkisebb AIC értékű modellt választjuk ki, mert valószínűleg az fog a legjobban teljesíteni. (Madaras 2024)

3.4. Neurális hálózatok

A mesterséges neuronhálók (ANN) lényege az emberi agyban található neurális hálózat működésének utánzása. Ezek olyan szoftveresen vagy hardveresen megvalósított, elosztott működésű rendszerek, amelyeket jellemzően sok, hasonló vagy azonos típusú, lokális adatfeldolgozást végző műveleti elemek, vagyis neuronok topológiája alkot.

A neuronhálók párhuzamosan épülnek fel és működnek, ezért rendkívül nagy számítási kapacitásra képesek. A neuronok közötti kapcsolatok irányított gráfként írhatóak le, ahol a hálózati csomópontok a neuronok. A kapcsolatokhoz súlyok tartoznak, amelyek a kapcsolat erősségét és irányát adják meg. Ezt a fejezetet Brassai (2019), valamint Russell és Norvig (2005) könyve alapján írtam.

Egy neuronháló általában háromféle rétegből épül fel:

- **Bemeneti réteg:** fogadja az adatokat vagy az információkat, és továbbítja azokat a háló többi részébe.
- **Rejtett rétegek:** a bemeneti adatokat feldolgozzák és összetettebb mintázatokat fedeznek fel az adatokban. Itt használunk a aktivációs függvényeket.
- **Kimeneti réteg:** a neuronháló kimenetét adja. Ez lehet egy előrejelzés, egy osztályozás vagy bármilyen más kimeneti forma, amely az adott problémától függ.

A neurális hálózatok működése két fő fázisból áll:

- **Tanulási fázis:** a hálózatban valamilyen módon eltároljuk a kívánt információfeldolgozó eljárást.
- **Előhívási fázis** (*recall*): a tárolt eljárás felhasználásával elvégezzük az információfeldolgozást.

A neuronhálók a tanulás során, az adatokból kapott visszacsatolás alapján a módosítják a paramétereiket (eltolási értékek és súlyok). Általában, a tanulási fázis lassú, több iterációt, sok sikertelen tanulási szakaszt hordozhat.

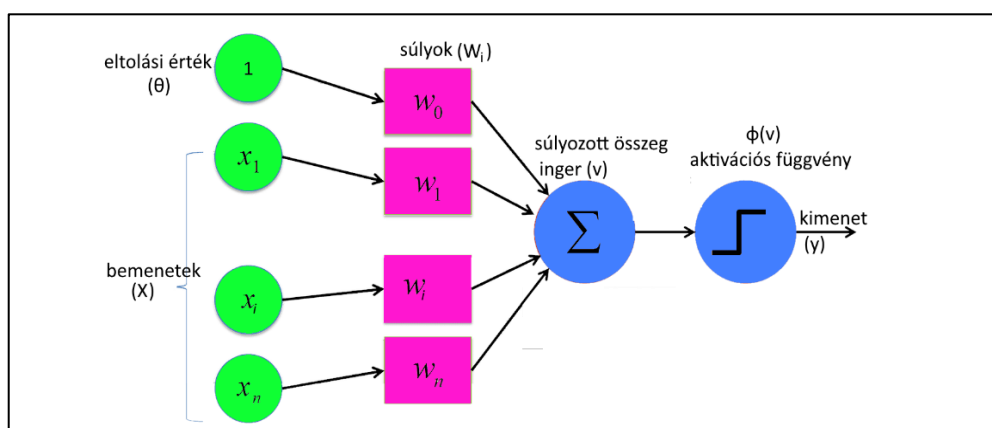
A tanítás korszakokra (*epoch*) van lebontva, és egy-egy epoch lefuttatása után a tanítási paramétereket újra lehet hangolni. Egy epoch során a háló teljesen végigmegy a tanító adatokon. Az előhívási fázisban a pillanatnyi bemeneti értékek alapján meghatározzuk a neuronháló kimenetét.

A tanítási folyamat során, ha van egy teszt adatbázisunk is a keresztvalidációra, akkor egyszerre számolhatjuk korszakonként a tanító- és teszt adatbázisra kapott veszteségek görbéit. A cél az, hogy a teszt adatbázison a legkisebb veszteség mellett fejeződjön be a tanulás, vagyis a modell a legjobban tudjon általánosítani, és ne tanítsuk túl.

3.4.1. Neuron, Perceptron, MLP

A neuron olyan feldolgozó elem, amely több bemenetet fogad és egy kimenetet generál. Az aktuális kimeneti értéket általában úgy adja, hogy a bemenetére kapott jelek súlyozott összegét egy (nemlineáris) aktivációs függvényben kiértékeli.

A perceptron egy régebbi típusú mesterséges neurális hálózat, egyrétegű előrecsatolt neurális hálónak is nevezik. Ebben a hálózatban csak egy feldolgozó egység található, ami általában lépcsőugrás aktivációs függvényt alkalmaz, amely egy adott küszöbérték felett, illetve alatt konstans kimenetet ad. Eredetileg Frank Rosenblatt javasolta egy olyan hálózatként, amely megfelelő beállítás és tanítás után képes szétválasztani két (lineárisan szeparálható) mintahalmazt. Ebből adódóan képes bemeneteket két osztályba sorolni, tehát egy lineáris osztályozó algoritmus.



3.2. ábra. Egy perceptron felépítése.

Forrás: saját szerkesztés Brassai (2019), 80. oldal alapján

Egy perceptron felépítését a 3.2. ábrán szemléltetem. Az egyszerű perceptron képtelen bonyolultabb feladatok megoldására, viszont a több perceptronból felépített hálók (többrétegű

perceptron- MLP) sokkal komplexebb feladatok elvégzésére is képesek, például szövegfelismerés, képfelismerés, regresszió és előrejelzés.

Ezek a hálózatok a ki- és bemeneti rétegen kívül egy vagy több rejtett réteget tartalmazhatnak. Az MLP egy előrecsatolt hálózat, tehát az információ egy irányba halad, nincsenek elemi visszacsatolások. Az egyes neuronok kimenete a vele összekapcsolt, következő rétegben levő neuron (egyik) bemenetét képezi. Egy perceptron kimenete a (3.12) képlettel írható le:

$$y = \varphi \left(\sum_{i=1}^N x_i w_i - \theta \right) \quad (3.12)$$

, ahol φ az aktivációs függvény y , x_i az adott bemenet, w_i a bementhez tartozó súly, θ az eltolási érték (bias). Amikor minden szomszédos neuron kapcsolódik egymáshoz, azt teljesen összekötött topológiának nevezzük. Az ilyen összetett hálózatok képesek a mély tanulásra (*deep learning*), amely során összetettebb mintázatokat és hierarchikus jellemzőket tanulhatnak meg.

3.4.2. Az MLP modell optimális szerkezetének kiválasztása

Heaton (2005) (127 – 130. oldal) leírta, hogy az MLP neuronhálóknál különböző problémák függvényében mennyi rejtett réteget érdemes használni, valamint kiindulópontokat fogalmazott meg arról, hogy az egyes rétegeknek hány darab neuront kellene tartalmazniuk. A bementi réteg annyi neuronból kell álljon, ahány (a kimenetet befolyásoló) független változónk van. A kimeneti rétegben pedig annyi neuron kell legyen, ahány kimenetet ad a neuronháló.

Két rejtett réteg használata mellett bonyolult, nemlineáris összefüggéseket is képesek vagyunk reprezentálni, feltéve, hogy legalább az egyik rétegben nemlineáris aktivációs függvény van. Mivel pontosan nem határozható meg előre az, hogy az adott problémára mennyi rejtett réteg szükséges, kezdetben érdemes egy-két rejtett réteggel kezdeni, majd folyamatosan növelni azok számát. A túl sok rejtett réteg vagy túl sok neuron könnyen vezethet a modell túltanításához.

Néhány hüvelykujj szabály szerint a rejtett rétegekben levő neuronok száma a ki- és bemeneti rétegben levő neuronok száma között kell legyen, vagy a bemeneti réteg neuronjainak számának kétharmad része plusz a kimeneti neuronok száma. A rejtett rétegek neuronjainak száma ne haladja meg a bemeneti neuronok számának kétszeresét.

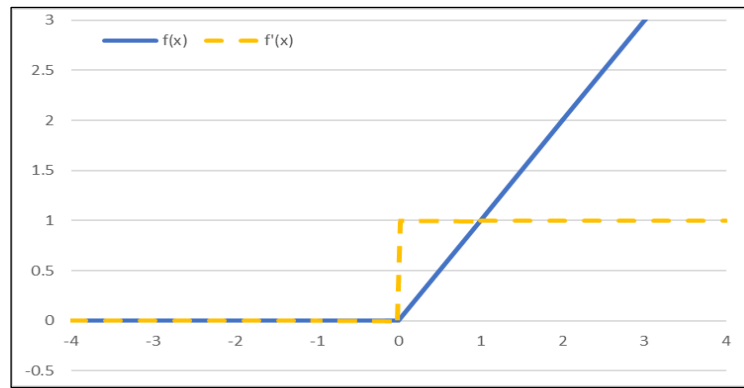
Sokféle optimalizációs algoritmus létezik, a Scikit-learn MLPRegressor osztály az SGD (Stochastic Gradient Descent), ADAM (Adaptive Learning Estimation és L-BFGS (Limited-memory Broyden–Fletcher–Goldfarb–Shanno) algoritmusokat támogatja. Általában az ADAM jónak bizonyul nagy adathalmazokra is, gyorsan tud konvergálni. A programom MLP részében bármely három algoritmust ki lehet választani a modellek készítése során.

3.4.3. ReLU aktivációs függvény

Az aktivációs függvények matematikai függvények, amelyek meghatározzák egy neurális hálózat rétegeinek kimenetét az adott bemeneti adatok alapján. Céljuk, hogy minden egyes neuronhoz egy aktiválási állapotot rendeljenek (aktív vagy inaktív).

Ez az állapot jelzi, hogy a neuron milyen mértékben járul hozzá a réteg kimenetéhez. Általában ezen függvényeknek nemlineárisnak és/vagy differenciálhatónak kell lenniük, mivel számos optimalizálási algoritmus a hálók súlyait gradiensek segítségével hangolja. A gradiens a függvény legnagyobb növekedésének irányát megadó vektor, ami a veszteségfüggvény parciális deriváltjaiból áll. Megmutatja, hogy a hibafüggvény milyen irányba lejt, milyen irányba menjen tovább az optimalizációs algoritmus annak érdekében, hogy az egyre kisebb hiba felé haladjon.

A legelterjedtebb transzferfüggvények közé tartozik például a logisztikus (szigmoid), hiperbolikus tangens (tanh), ReLU, Gauss, lépcsőugrás függvény, telítéses lineáris függvény. A kutatás során a ReLU aktivációs függvényt használtam mindegyik idősorra, mivel a szakirodalomban ez a legelterjedtebb. Bamberger és társai (2023) kifejtették, hogy bizonyos jelenségek approximációja esetén milyen aktivációs függvényeket érdemes használni. A munkanélküliségi ráta függvénye a munkanélküliek számának és a munkaerő-állománynak a hányadosa, tehát a függvény nulladrendű homogén, vagyis skála-invariáns. Ebből adódóan Bamberger és társai (2023) (7. oldal, 2.5-ös tétel) szerint, az ilyen függvények esetében a ReLU aktivációs függvény használható.



3.3. ábra. a ReLU grafikonja.

Forrás: saját szerkesztés.

A ReLU (Rectified Linear Unit – egyenirányított lineáris egység) az érkezett ingert adja vissza, ha nullánál nagyobb, különben nullát ad vissza, tehát a neuron inaktív lesz (3.13).

$$\varphi(x) = \max(0, x) \quad (3.13)$$

Rendkívül gyors és kicsi a számításigénye: Deriváltja a (nullán kívül) mindig 1. A 0 kimenetet generáló neuronok kihagyhatóak neuronhálóból, csökkentve a számításigényt, és nem okoz gradiens-elhalást. Sok rétegből álló (mély) neuronhálók esetében sokkal jobb, mint például a szigmoid vagy a tangens hiperbolikus függvény.

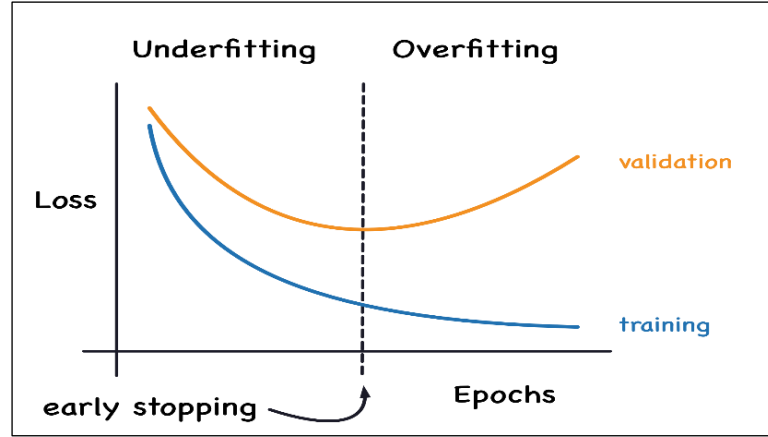
A gradiens-elhalás probléma akkor fordul elő, amikor a gradiensek túl kicsik lesznek, és emiatt a súlyokat nehéz vagy lassú frissíteni a tanulás során.

3.4.4. Ellenőrzött tanítási stratégia

A neuronhálók tanítása egy többváltozós optimalizációs folyamat, egy előre meghatározott célfüggvény (költség- vagy veszteségfüggvény, például MSE) alapján. A legtöbb optimalizációs eljárás a gradienseket figyeli legkisebb veszteség eléréséhez.

Én a felügyelt, más szóval az ellenőrzött tanulás (supervised learning) módszerét alkalmaztam, amely során egy előre megadott tanító adatbázist adunk meg a neuronhálónak. A tanító adatbázis bemeneti és az ahhoz tartozó elvárt kimeneti adatok (target) párjaiból áll. A hiba-visszaterjesztéses (backpropagation) algoritmus célja az, hogy a modell a bemenetek és kimenetek közötti kapcsolatokat megtanulja úgy, hogy minimalizálja a modell becslései és az elvárt kimenet közötti hibákat. Ilyenkor a kimeneti rétegtől visszafelé haladva, rétegről rétegre kiszámítja a költségfüggvény gradiensét, majd annak alapján, a kisebb hiba irányába való haladás érdekében frissíti a hálózat súlyait és eltolási értékeit.

A modellek tanítása során gyakran előforduló probléma lehet a túltanítás (overfitting). Ilyenkor a tanítási korszakok növelésével a hiba csökken a tanító adatok halmazán, viszont a validációs vagy teszt halmazban levő elvárt kimenetektől egyre távolodnak a modell becslései.



3.4. ábra. A tanulási és validációs folyamatok veszteség görbéi.
Forrás: (Holbrook & Cook, 2023)

Ehhez sokféle tényező hozzájárulhat, például a nem megfelelő mennyiségű vagy minőségű tanítóadat, a túl sok tanítási ciklus (epoch), nem megfelelő hálószerkezet és a rosszul beállított hiperparaméterek is okozhatják.

A túltanítás ellen többek között a keresztvalidáció és a korai leállítás segíthet. A keresztvalidáció során az adathalmazt több részre osztjuk, és különböző részeken teszteljük a modellt, hogy ellenőrizzük, mennyire jól általánosít az új, ismeretlen adatokra.

A gradiens süllyedéssel algoritmusoknál a tanítóadatokat összekeverhetik minden iteráció után (shuffle), hogy csökkentsék a mintavételi elfogultságot.

Emellett valamilyen regularizációt is szoktak használni, hogy büntessük a bonyolultabb szerkezeteket és a túl nagy súlyokat, előnyben részesítve a hasonló megoldást adó, de egyszerűbb szerkezeteket, és így csökkentve a túltanulás esélyét.

A dolgozatomban használt MLPRegressor modell L2 regularizációt (Ridge regresszió, vagy weight decay) használ, amely lényege, hogy a hagyományos veszteségfüggvényhez hozzáadunk egy büntetési tagot, amely a modell súlyainak négyzetes összegét tartalmazza. A büntetés mértékét egy λ (0 és 1 közötti) súllyal szabályozhatjuk. A (3.14) képletben például az MSE veszteségfüggvényhez adjuk hozzá a súlyok négyzetösszegét (regularizációs tag).

$$L(W) = MSE(W) + \lambda \sum_{i=1}^n w_i^2 \quad (3.14)$$

3.4.5 Optimalizációs algoritmusok

A legelterjedtebb optimalizációs algoritmusok közé tartozik a sztochasztikus gradiens süllyedéssel (SGD) algoritmus, ami iteratíván haladva hangolja úgy a súlyokat, úgy, hogy az egyre kisebb hiba (pl. MSE) felé haladjon. Az egyes iterációk során véletlenszerűen kiválasztott minták (batch-ek) alapján történik a frissítés, ami hatékonyabbá teszi a nagy adatkészletek feldolgozását.

A tanulási ráta meghatározza, hogy a gradiens irányában milyen lépésnagysággal frissítjük a modell súlyait. Ha a tanulási ráta túl magas, az optimalizáló túllépheti a minimumot, ha túl alacsony, akkor pedig lassan fog konvergálni vagy elakadhat helyi minimumokban. Mivel hiperparaméternek számít, általában kísérletezéssel állítják be (pl. 0.01 és 0.001 között). Az SGD úgynevezett momentumot is használ, hogy elkerülhesse a lokális minimumokban való elakadást. A momentumos tagnak a lényege, hogy figyelembe veszi a súlytényezők módosításának az irányát az előző lépésből.

Az ADAM (Adaptive Moment Estimation) adaptív tanulási rátákat használ minden egyes paraméterhez. Ez azt jelenti, hogy minden paraméterhez külön tanulási ráta tartozik, amely az algoritmus által automatikusan kerül beállításra az adott paraméterhez képest, a gradiens mozgóátlagai alapján. Az ADAM az első és második momentumokat (a gradiens és azok négyzeteinek mozgóátlagait) kombinálja az exponenciális mozgóátlagok segítségével.

Az L-BFGS (Limited-memory Broyden-Fletcher-Goldfarb-Shanno) egy kvázi-Newton módszer, és korlátozott memóriát használ a Hesse-mátrix közelítésére, így kevésbé memóriaigényes, mint a teljes Hesse-mátrix számítása. A Hesse-féle mátrix egy többváltozós függvény másodrendű parciális deriváltjaiból álló négyzetes mátrixa. Az L-BFGS előre kiszámítható és determinisztikus, így stabilan működik különböző problémákon.

Az optimalizációs algoritmusok kiválasztása több tényezőtől függ, például a feladat jellegétől, az adatok méretétől, a modell architektúrájától és az elérhető számítási erőforrásoktól. A dolgozatomban, az MLP modellek tanítására mindhárom algoritmust kipróbáltam. Az ADAM és SGD esetében a programommal kiszámítom és megjelenítem az MLP modellek által elért tanulási és validációs veszteség görbéket, amely segít kiszűrni a túltanulást is.

4. A kutatási gyakorlati része

Ebben a fejezetben a kutatás gyakorlati részét tárgyalom. Először a kutatáshoz készített programom működését ismertetem, majd a benne végzett számítások és tesztek eredményeit.

4.1. A Django webalkalmazás működése

A Django egy Python-alapú webes keretrendszer, amely lehetővé teszi a gyors és biztonságos weboldalak fejlesztését. Az MVC (Model-View-Controller) szemléletet követi. Controller fájlok helyett a views.py fájlban található függvények segítségével dolgozzuk fel az adatokat és generáljuk a nézeteket, és küldjük vissza a választ a kliensnek. Az URL címeket az urls.py fájlban regisztráljuk, amelyekhez hozzárendeljük controller függvényeket, a kérést átadva paraméterként. A modellek az alkalmazás adatstruktúráját és az adatbáziskezelést definiálják. A sablonok (template) segítségével határozzuk meg az oldalak elrendezését és tartalmát, amelyek dinamikusan generálhatók és jeleníthetők meg az adatok alapján a weboldalon (Dhandi, 2024).

A Django keretrendszer egy virtuális környezetben fut, külön Python értelmezővel, amely egyrészt biztonságos, másrészt egy szöveges fájlba automatikusan hozzáadódnak a függőségek (kiegészítő Python csomagok), amelyek a projekt működéséhez szükségesek, így más számítógépen is könnyen be lehet üzemelni a projektet, csak néhány fájlt kell megőrizni (pl. manage.py, settings.py, urls.py, views könyvtár, models.py, requirements.txt, wsgi.py), illetve néhány parancsot kell lefuttatni a kiszolgáló gépen (virtuális környezet reprodukálása).

A django alapvetően az Nginx webszervert használja, de a WSGI-nek köszönhetően más kiszolgálóval is össze lehet kötni. A WSGI (Web Server Gateway Interface) egy szabványos interfész, amely lehetővé teszi a webszerverek számára, hogy a Python programok felé továbbítsák a HTTP kéréseket és válaszokat.

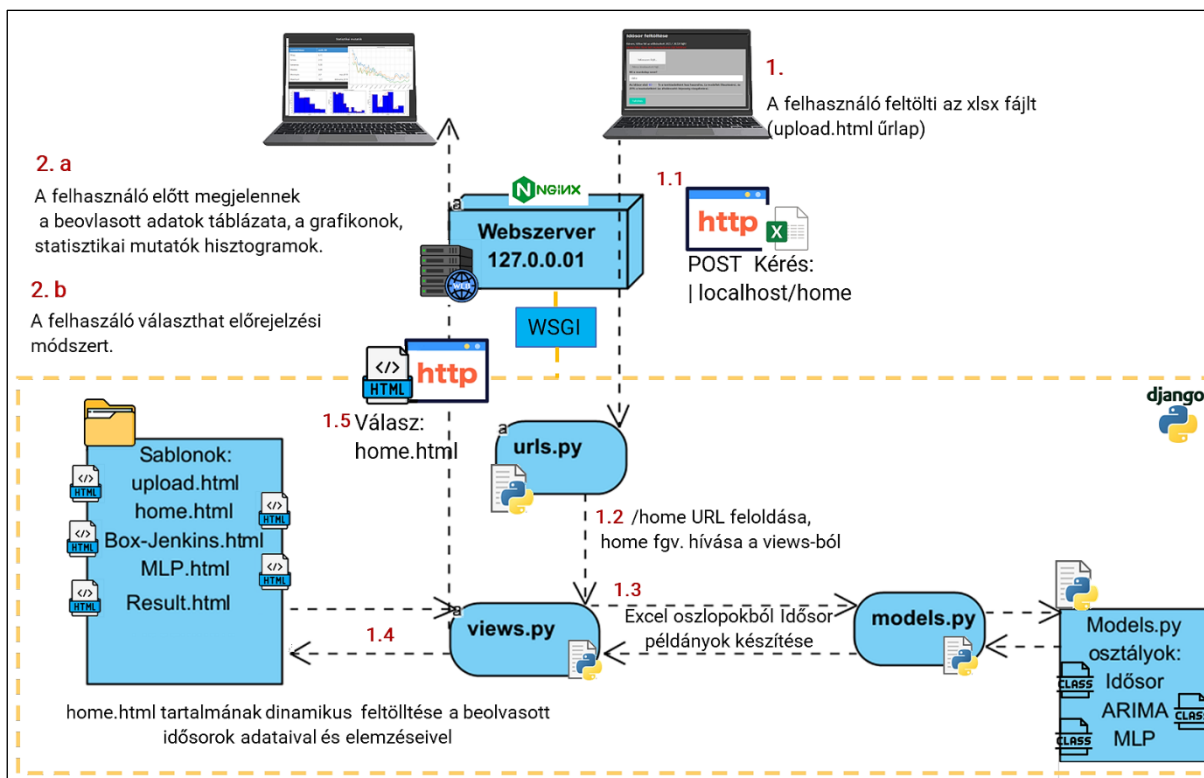
A webszervert parancssorból lehet elindítani, viszont előtte mindig aktiválni kell a virtuális környezetet (4.1. ábra).

```
C:\Users\Kiki>cd /d C:\AllamVizsga\venv\Scripts & activate & python
C:\AllamVizsga\venv\AllamVizsga\manage.py runserver
Watching for file changes with StatReloader
Performing system checks...

System check identified no issues (0 silenced).
May 26, 2024 - 11:26:18
Django version 5.0, using settings 'AllamVizsga.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.
```

4.1. ábra: A Django projekt elindítása konzolból (saját ábra).

A 4.2. ábrán az alkalmazás logikai felépítése látható. Mindegyik sablonban a Bulma CSS keretrendszer osztályait használtam fel a minimalista oldalszerkezet kialakítására. Igyekeztem reszponzívra kialakítani az oldalakat, mobilnézetben (600px) alatt az ábrák egymás alá kerülnek, illetve a táblázatok átrendeződnek.



4.2. ábra: A fájlfeltöltés folyamata és a webalkalmazás logikai felépítése (saját ábra).

4.1.1 Adatbeolvasás

A letöltött és Excel táblázatban rendszerezett idősorokat egy űrlapon keresztül lehet feltölteni (4.3. ábra). Az idősorokat a Pandas nevű Python modul segítségével olvasom be DataFrame adatszerkezetekbe. A beolvasott adatszerkezetekből létrejönnek az idősorok számára készített osztálypéldányok.

The screenshot shows the web application's main page, which includes a file upload form. The form has a "Válasszon fájlt..." button, a text input field for the file name (data.xlsx), and a text input field for the user's name (Maros). Below the form, there is a message indicating that the first 93% of the data is used for training and the remaining 7% is used for testing. A green "Feltöltés" button is located at the bottom of the form.

4.3. ábra: A webalkalmazás kezdőlapja (állomány feltöltő űrlap).

Egy idősor nevű osztályban van tárolva az idősor neve, a tanító- és tesztadatok listái, statisztikai mutatói, grafikonok, illetve a számításokat végző függvények és eljárások. Egyszerre több idősort is képes beolvasni a program, feltéve, hogy azok egymás mellett elhelyezett, egyenlő számú sorral rendelkező oszlopokba vannak rendezve. Az első sor fejléc, amely tartalmazza az idősorok megnevezéseit. Az első oszlop a megfigyelések időpontjait vagy sorszámait kell tartalmazza (4.1. táblázat).

időszak	Kovácsna	Hargita	Maros
2010 január	12.2	10.9	8.3
2010 február	12.5	11.4	8.4
2010 március	12.3	11.2	8.5

4.1. táblázat. Példa az idősorokat tartalmazó Excel állomány formázására.

Forrás: INNSE

Az idősor állományt feltöltő oldalon egy JavaScript kóddal, eseményfigyelőkkel ellenőrzöm a tanítóhalmaznak megadott arányt: A teljes idősor legalább 50%-a kell legyen tanítóadat, de nem lehet több, mint az adatsor 95%-a. Ezt szerveroldalon is ellenőrzöm természetesen.

A sikeres beolvasást követően a program egy HTML táblázatban megjeleníti a beolvasott adatokat, valamint azokat egy közös grafikonon is ábrázolja (4.4. ábra). Természetesen egyetlen idősort is fel lehet tölteni. A program a feltöltött idősorokról hisztogramokat készít, és lefuttatja rajtuk a Kolmogorov–Smirnov tesztet a normális eloszlásra, amelyek eredményét (p-érték és statisztika) egy táblázatban foglalja össze. Erről az oldalról van lehetőség előrejelzési módszert választani, valamelyik gombra kattintva: Box-Jenkins eljárás, vagy MLP neuronhálók.

4.1.2. A beolvasott idősorok tárolása munkamenet segítségével

A feltöltött idősorokat jelenleg nem tárolom permanensen, hanem csak a szerveret futtató számítógép memóriájában, objektumok formájában. A különböző felhasználók által feltöltött idősorokat egy globális szótárban tárolom, amelynek kulcsai a munkamenet azonosítók, így kiküszöbölve azt, hogy két felhasználó által feltöltött adatsor felülírja egymást. Az értékek pedig az idősorok példányait tartalmazó listák.

Minden felhasználóhoz, aki a webalkalmazásomat eléri (kommunikál az azt futtató szerverrel) tartozik egy munkamenet (session). A webfejlesztésben a munkamenet egy olyan mechanizmus, amely lehetővé teszi, hogy adatokat tároljunk egy felhasználó számára több kérés során. Ezáltal az információkat meg tudjuk őrizni különböző oldalak között anélkül, hogy

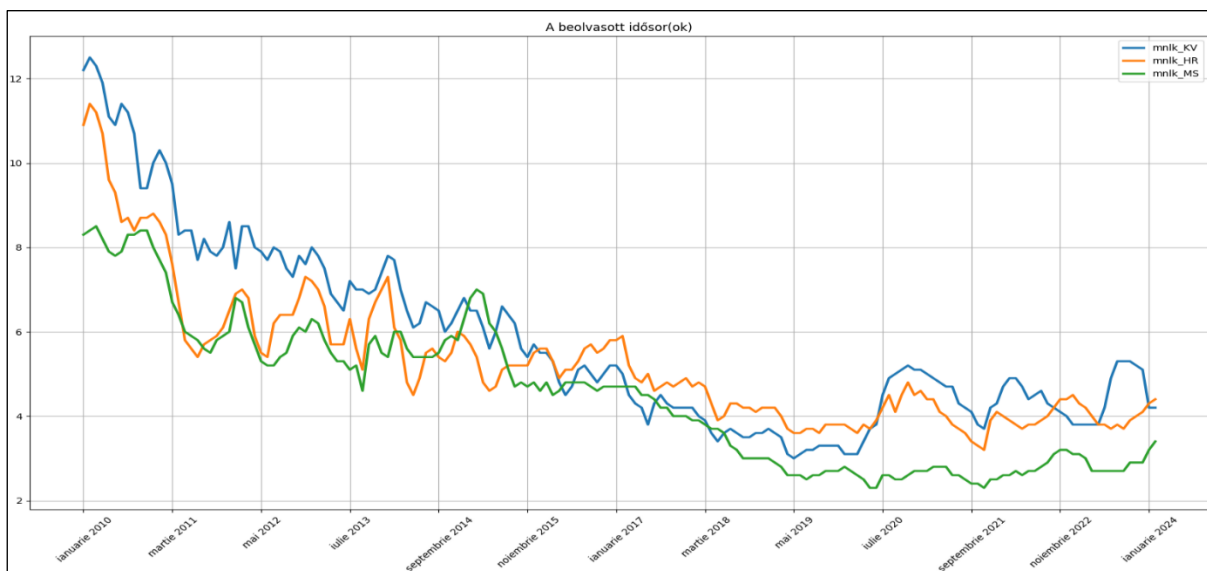
a felhasználónak újra és újra meg kellene adnia azokat. A munkameneteket gyakran használják a felhasználói hitelesítés, a bevásárlókosarak és más felhasználóspecifikus adatok kezelésére. A Django-ban egy munkamenet a böngésző bezárásáig tart.

Ezzel megoldottam azt, hogy a fájl feltöltését követően, a felhasználó előre és vissza is tudjon navigálni a weboldalon anélkül, hogy újra fel kellene töltsé az adatsort.

A következő fejezetben a programot felhasználva mutatom be Kovászna, Hargita és Maros megye munkanélküliségi rátáinak elemzését és modellezését.

4.2. Az idősorok elemzése

Az előrejelzések készítése előtt fontos megvizsgálni az idősorok alapvető statisztikai tulajdonságait, mivel iránymutató szerepe lesz a prediktív modellek készítése során.



4.4 ábra. A program által generált grafikon a beolvasott idősorokról.
Forrás: saját ábra, Pyplot segítségével, az INNSE adatai alapján.

Az idősorok grafikonján (4.4 ábra) látszik, hogy Maros megyében szinte végig a legalacsonyabb a munkanélküliségi ráta, amely többek között a megye ipari fejlettségének, ezáltal a több munkahelynek is tulajdonítható. Összességében 2020 júniusáig csökkenő trend volt megfigyelhető.

Sok periódusban megfigyelhető, hogy télen magasabb volt a mutató, mint a többi évszakban, ez például a szezonális munkákhoz (pl. mezőgazdaság és építkezések) is köthető, viszont a pontos okok tisztázása egy külön kutatást igényel és nem tartozik ezen dolgozat témájához.

Feltűnő, hogy Kovászna megyében 2020 júliusa óta szélsőségesebben változik a mutató, mint a másik két régióban. Ezt a modelleknek is nehezebb volt egyébként „megtanulni”, nem meglepő, hogy Kovászna megye esetében a modellek gyengébben teljesítettek a tesztelő adatbázison, mint Hargita és Maros megye adatain.

mutató	Kovászna megye	Hargita megye	Maros megye
Átlag	5.89	5.37	4.62
Szórás	2.25	1.63	1.73
Variancia	5.05	2.65	2.99
Medián	5.15	5.1	4.7
Minimum	3	3.2	2.3
Maximum	12.5	11.4	8.5

4.2. táblázat. A beolvasott idősorok statisztikai mutatói. Az adatok százalékban értendők.
Forrás: saját szerkesztés, az INNSE adatai alapján.

Kovászna és Hargita megyében a vizsgált időszakban 2010 februárjában volt a legmagasabb a munkanélküliségi ráta, előbbi esetében 12.5%, utóbbinál pedig 11.4%. Ezzel szemben Maros megyében 2010 márciusában érte el csúcsát a munkanélküliség, 8.5%-os értékkel. Ez a növekedés bizonyára a 2008-ban kialakult pénzügyi és gazdasági világválság hatása volt, amely kissé késve, de elérte Romániát is. Az ország gazdasága jellemzően az alacsony és közepes képzettségű munkaerőt használó, viszonylag kevés technológiát felhasználó és alacsony hozzáadott értékű iparágakon alapult. A gazdasági recesszió miatt rengeteg munkahely szűnt meg, vagy jelentősen csökkentette dolgozóinak létszámát (Georgeta, 2015)

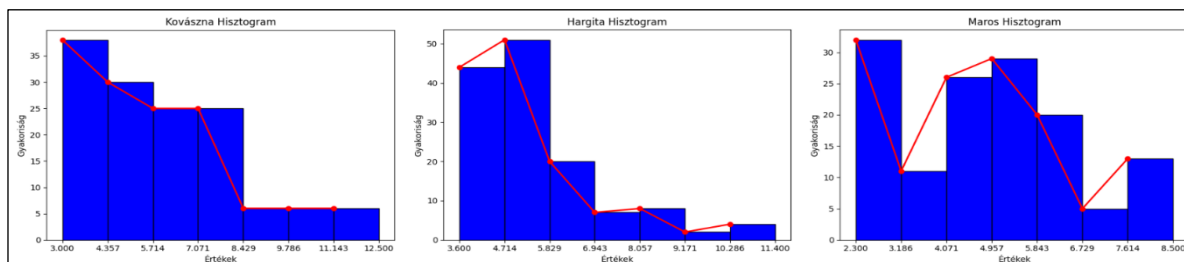
A vizsgált időszak legalacsonyabb munkanélküliségi rátája Maros megyében 2020 májusában volt, mindössze 2.3%. Hargita megyében a legalacsonyabb értéket 2021 novemberében mérték, amely 3.2% volt, míg Kovászna megyében 2019 májusában érte el a legalacsonyabb szintet, 3%-kal. Sajnos a koronavírus-járvány miatt 2020 tavaszától 2021 novemberéig jelentős ütemben növekedett a munkanélküliek száma, amelyhez a járvány visszaszorítására irányuló intézkedések is nagymértékben hozzájárultak. Az, hogy Maros megyét miért érintette kevésbé a koronavírus-járvány, egy külön kutatás keretében lenne érdemes megvizsgálni.

Kovászna megyében 2020 júliusától kezdődően, a másik két régióhoz viszonyítva, nagymértékű növekedések jellemezték az idősorokat a téli hónapok tekintetében. Ez arra utalhat, hogy a téli időszakban fokozódott a munkanélküliség, valószínűleg szezonális munkák megszűnése vagy csökkenése miatt.

4.2.1. Az adatok eloszlása

A programom az idősorok hisztogramjait is automatikusan elkészíti, valamint lefuttatja rajtuk Kolmogorov-Smirnov tesztet a normál eloszlásra, a scipy.stats Python modul kstest függvényének segítségével.

Jelen esetben én 7 darab oszlopra (tartományra) osztottam fel az adatokat. Az idősorokról készített hisztogramok a 4.5. ábrán láthatóak.



4.5. ábra. Az idősorok hisztogramjai.

Forrás: saját ábra, Pyplot segítségével, az INNSE adatai alapján.

Látható, hogy Kovászna és Hargita megye idősorainak eloszlásának alakja jobbra elnyúló, tehát a kisebb értékekből több van. A normalitás vizsgálatához Lázár (2022) alapján a Kolmogorov-Smirnov tesztet is elvégeztem, a scipy.stats csomag kstest függvényének segítségével. A D statisztika két vizsgált minta eloszlása közötti eltérést adja meg. Mivel a minták mérete 170, így a legnagyobb elfogadható D érték $\frac{1.38}{\sqrt{170}} = 0.104$ lenne, azonban mindhárom idősornál ez jóval nagyobb, valamint a p-értékek kisebbek, mint $\alpha = 0.05$, így elutasítjuk a nullhipotézist, miszerint a vizsgált idősor eloszlása nem különbözik a normális eloszlástól, tehát egyik megye adatai sem normális eloszlásúak.

Idősor	p-érték	D statisztika	statisztika	ferdeség
Kovászna	0.00	0.15	1.00	1.06
Hargita	0.00	0.13	1.00	1.59
Maros	0.00	0.16	0.99	0.54

4.3. táblázat. Az idősorokon elvégzett Kolmogorov-Smirnov tesztek eredményei, 5%-os szignifikancia szinten.

Forrás: saját szerkesztés, az INNSE adatai alapján.

A ferdeség (skewness) az adatok eloszlásának aszimmetriáját mutatja. Ha a ferdeség értéke pozitív, az eloszlás jobb oldala hosszabb, ami azt jelenti, hogy az adatok többsége a bal oldalon koncentrálódik.

4.2.2. A stacionaritás vizsgálata

Az idősorok stacionaritását az egységgyökér tesztekkel vizsgáltam, Madaras és Györfy (2016) alapján, az ADF és KPSS teszteket felhasználva. A gyakorlatban, ehhez a Python statsmodels.tsa.stattools csomagjából az adfuller és kpss függvényeket használtam fel. Az eredeti idősorok mellett az első differenciált értékeken is elvégeztem a teszteket. Az eredmények a 4.4. táblázatban találhatóak. A táblázatban látható értékek 5%-os szignifikancia szinten értendők.

Idősor	Eredeti idősor				Első differencia			
	ADF teszt		KPSS teszt		ADF teszt		KPSS teszt	
	Statisztika *	p-érték	Statisztika **	p-érték	Statisztika *	p-érték	Statisztika **	p-érték
Kovászna	-2.73	0.07	1.57	0.01	-3.44	0.01	0.44	0.06
Hargita	-2.7	0.07	1.5	0.01	-2.79	0.06	0.22	0.1
Maros	-2.51	0.11	1.69	0.01	-6.63	0	0.12	0.1

*ADF kritikus érték: -2.88, **KPSS kritikus érték: 0.46

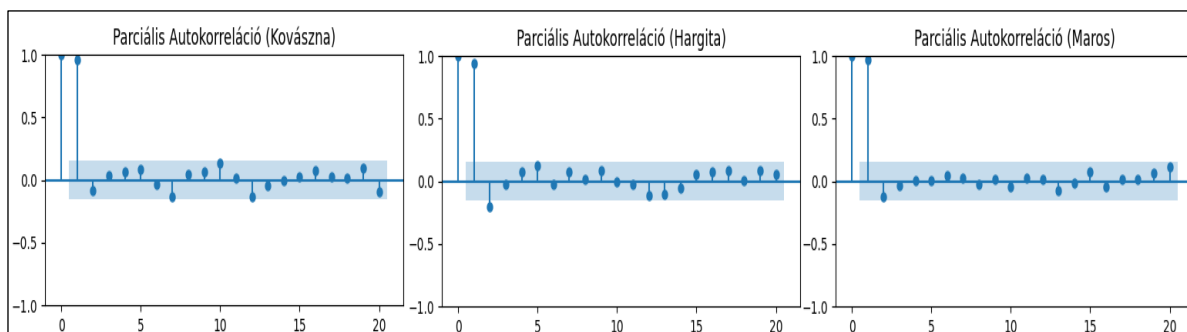
4.4. táblázat. az ADF és KPSS tesztek eredményei.
Forrás: saját szerkesztés, az INNSE adatai alapján.

Mindegyik megye idősorának esetében az ADF teszt p-értéke nagyobb, mint 0.05, valamint a statisztikák értékei nagyobbak, mint a kritikus érték (-2.88), tehát nem utasítjuk ez a nullhipotézist, vagyis az idősorok nem stacionáriusok. A KPSS tesztek p-értékei kisebbek, mint 0.05, valamint a statisztika értékek nagyobbak, mint a kritikus érték (0.46) tehát elutasítjuk a nullhipotézist, vagyis az idősorok eszerint sem stacionáriusok.

Szükség van a differenciálásra. A $d = 1$ (first differenced) esetében (4.4. táblázat jobboldala) már azt látjuk, hogy Kovászna és Maros megyénél, az ADF teszt statisztika értékei kisebbek, mint a kritikus érték (-2.88), és a p-értékek kisebbek, mint 0.05, tehát elutasítjuk a nullhipotézist, vagyis az idősorok most már stacionáriusok. Emellett látszik az is, hogy Hargita megye esetében is minimálisak a különbségek. A KPSS teszteknel mindhárom differenciált idősornál 0.05-nél nagyobb p-értéket, valamint 0.46-nál kisebb statisztikát kaptunk, tehát nem utasítjuk el a nullhipotézist, így az adatsorok stacionáriusok. Emiatt már nem kell tovább növelni a d értékét.

Az $MA(q)$, vagyis a mozgóátlag komponens rendjének kezdetleges meghatározásához elkészítettem az idősorok autokorrelációs függvényeit a statsmodels.tsa.stattools.acf

függvénnyel, és ábrázoltam azokat a pyplot modul segítségével. Az vizsgált idősorok autokorrelációs függvényei a 4.6-os ábrán láthatóak.



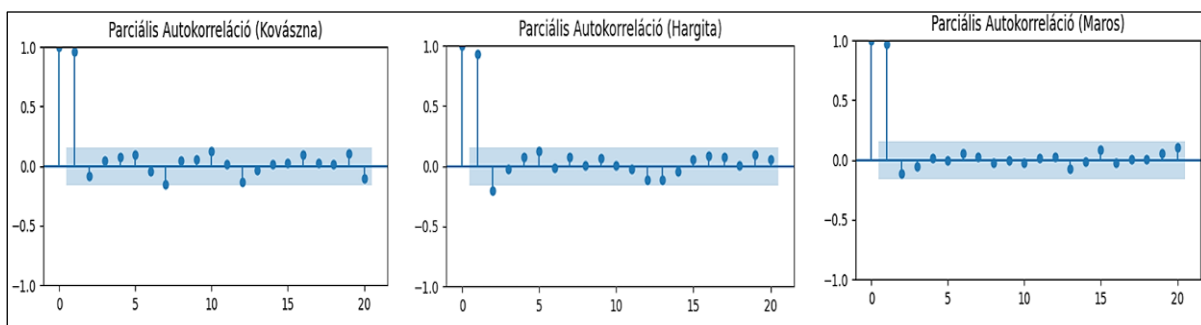
4.6. ábra. Az idősorok autokorrelációs függvényei.

Forrás: saját szerkesztés Pyplot segítségével, az INNSE adatai alapján.

A kék beszínezett tér a vonalak mögött a 95%-os konfidencia intervallum, ami azt jelzi, hogy hol várható az autokorrelációs együtthatók értéke egy adott szinten. Ha egy autokorrelációs együttható a kék színnel befestett területen belül helyezkedik el, akkor azt mondhatjuk, hogy nincs jelentős autokorreláció az adott késleltetett értékben (Drelczuk, 2020).

Ha az adott idősor egy $MA(q)$, vagyis mozgóátlag folyamat lenne, akkor az első néhány k lépés után a p_k autokorrelációk hirtelen csökkennének (Sándor, 2019), viszont a 4.6. ábrán levő ACF-ek nem ezt mutatják, hanem lineáris, fokozatos csökkenést. Ebből adódóan az autokorrelációs függvénnyel szerint az idősorok nem $MA(q)$ folyamatok. Azonban ez önmagában még jelenti azt, hogy az $ARMA(p, q)$, vagy $ARIMA(p, d, q)$ modellek ne jöhetnének szóba.

Az idősorok $AR(p)$ autoregresszív komponenseinek meghatározásához szükség van a parciális autokorrelációs függvényekre, amelyek a 4.7. ábrán láthatóak. Az autokorrelációs függvényeket a `statsmodels.graphics.tsaplots` modul `plot_acf` függvény segítségével ábrázolom, egy közös grafikonon.



4.7. ábra. A parciális autokorrelációs függvények diagramjai.

Forrás: saját ábra, Pyplot segítségével, az INNSE adatai alapján.

Mindhárom megye esetében az látszik a parciális autokorrelációs függvényen, hogy az első kettő lépésben az autokorrelációs együtthatók egyhez közeli értékek, míg az összes többi lépésben elhanyagolhatóan kicsik. Ebből adódóan AR (2) modellel érdemes kezdeni, de szóba jöhetnek az ARMA vagy ARIMA típusú modellek is, mivel a PACF önmagában nem tudja meghatározni vagy kizárni az ARMA (p, q), ARIMA (p, d, q) folyamatokat, így többféle modellt is leteszteltem (Sándor, 2019).

4.3. Az ARIMA modellek implementációja

A 4.4. táblázat és a 4.6-os, 4.7-es ábrán levő grafikonok azt mutatják, hogy valószínűleg mindhárom megye esetében az ARIMA (2, 1, 0) modellel érdemes próbálkozni első körben a becslések készítéséhez, viszont megvizsgáltam még például az ARIMA (2, 1, 1), ARIMA (2, 1, 2) modelleket is, és ezek közül kiválasztottam a legkisebb AIC (*Akaike Information Criterion*) értékű modellt a Sándor (2019) alapján, mert valószínűleg ez a modell fog a legjobban illeszkedni az adott idősorra.

A Python programban a statsmodels.tsa.arima.model.ARIMA ingyenesen telepíthető csomag segítségével illesztettem a modelleket és készítettem el az előrejelzéseket, Brownlee (2023) alapján. A felhasználó, a feltöltést követően, a „Box-Jenkins eljárás” gombra kattintva a 4.8. ábrán látható űrlapra lesz irányítva, amely fölött az ACF, PACF diagramok, az ADF és KPSS tesztek is megjelennek.

Minden egyes feltöltött idősorhoz természetesen egy külön űrlap (form) tartozik, amelyek dinamikusan kerülnek generálásra (a feltöltött idősoroknak megfelelően). Ha jelölőnégyzetet kijelöljük, akkor az autoARIMA modul megkeresi a legkisebb AIC értékkel illeszkedő modellt.

Az illesztett modellek komponenseihez kiszámolt együtthatókat természetesen megjeleníti számunkra a program.

A legjobb p, d, q paraméterek automatikus megkeresése AIC alapján ☐

p
2

d
1

q
2

Kért előrejelzések száma a tesztadatokon túl
6

ARIMA előrejelzések készítése

4.8. ábra: az ARIMA modellek lekérését szolgáló űrlap (saját ábra).

A kék gombra (4.8. ábra) kattintva a modellek tanítása, tesztelése és az előrejelzések készítése automatikusan megtörténik. Ez időigényes folyamat lehet, az idősorok méretétől és a modellek bonyolultságától függően. Ezalatt a felhasználó egy betöltési animációt lát. Amint megvannak az eredmények, a webszerver egy összesítő oldalra irányítja a felhasználót, ahol a tanulási, tesztelési és előrejelzési grafikonok és táblázatok láthatóak.

4.3.1 A modellek illesztése

Az AIC értékeket a Python-ben létrehozott statsmodels.tsa.arima.model.ARIMA osztállyal készített modellek *model_fit*, *aic* attribútumából kértem le, a tanítóadatokra való illesztést követően, amelyek a 4.5. táblázatban vannak modellenként összefoglalva.

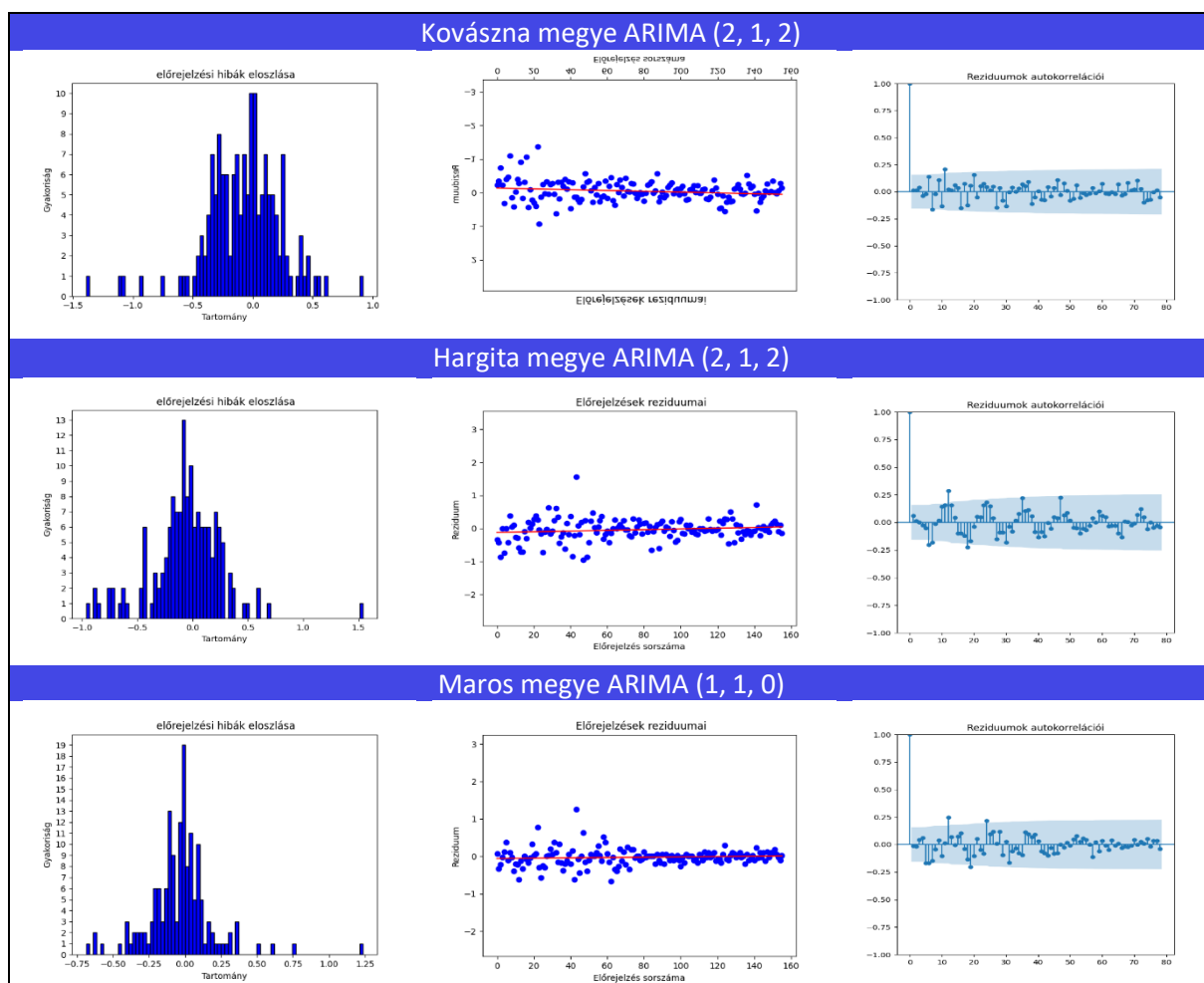
Megye	Modell	AIC	MSE	RRMSE	MAPE	R ²
Kovászna	ARIMA (2, 1, 2)	91.51	0.10	0.05	3.97	0.98
Hargita	ARIMA (2, 1, 2)	102.17	0.11	0.06	4.23	0.96
Maros	ARIMA (1, 1, 0)	-10.9	0.05	0.05	3.24	0.98

4.5. táblázat. Az illesztett ARIMA modellek teljesítménye a tanító adatokon (saját szerkesztés).

A kutatás során több modell is ki lett próbálva, az AutoARIMA rácskeresés segítségével: ARMA ($p=1..5$, $q=1..5$), ARIMA ($p=1..3$, $d=1..3$, $q=1..3$). A legkisebb AIC értékekkel illeszkedő modelleket szintén az 4.5. táblázatban írtam le. Az első differencia ($d = 1$) esetében az idősorok már stacionáriusok voltak (4.4. táblázat). Az egynél több differenciálás emellett jelentősen rontotta a modellek illeszkedését és általánosító teljesítményét.

Az illesztés után a programban kiszámítja a modelleknek a tanítóadatokra adott becsléseit, és ezek alapján kiszámítja a hibamutatókat és a determinancia-együtthatót. A becslések reziduumaikat a 4.9. ábrán látható formában jeleníti meg a program.

A hisztogramokon látszik, hogy a hibák normális eloszlást követnek. A pontdiagram (scatter plot) alapján kivehető, hogy a hibák nagyjából egy egyenes mentén helyezkednek el. Azonban mindhárom modell esetében, a becslési hibák autokorrelációit szemléltető diagramokon megfigyelhető, hogy több késleltetett értéknél is szignifikáns autokorrelációs kapcsolat van. Ez azt jelenti, hogy a prediktív modell nem magyarázza teljes mértékben az idősor összes mintázatát, van egy fennmaradó szerkezet vagy mintázat a reziduumban, amit a modell nem tudott megfogni. A szezonális ARIMA (SARIMA) modellekkel ezt ki lehetne küszöbölni, azonban jelen dolgozat a Box-Jenkins eljárással készült ARIMA modelleket vizsgálja.



4.9. ábra. Az ARIMA modellek tanító adatokra adott becsléseinek reziduumaik.

Forrás: saját ábra, Pyplot segítségével, az INNSE adatai alapján.

4.3.2 A modellek tesztelése

A modellek illesztését követően megnéztem, hogyan általánosítanak, vagyis mennyire pontosan becsülik meg a tesztadatok értékeit, MSE, RRMSE, MAPE és R2 mutatókat kiszámítva és figyelembe véve. Ehhez Brownlee (2023) alapján, az előrelepő (walk-forward) validációt alkalmaztam.

Ilyenkor a modell kezdetben a tanítóadatokra illeszkedve, egy lépésben becsül egy értéket, majd, a tesztadatokból megkapja a valódi megfigyelést, és azzal újra illeszti önmagát. A becsléseket egy listában tárolom. A becslések és az előrejelzések készítésének függvénye az 4.10. ábrán látható.

Ez a módszer azért jobb, mint egy lépésben több értéket becsülni, mivel abban az esetben nagyjából csak egy egyenest kapnánk, így viszont a valósághoz közeli értékeket kapunk. Az előrelepő validáció folyamatos frissítéssel biztosítja, hogy a modell mindig a legfrissebb adatokra támaszkodjon, így jobban tud alkalmazkodni az időbeli változásokhoz.

```
def rollingForecast(self, train_data:list, test_data: list, mode: str = "test", n_pred: int = 6):
    predictions = []
    if mode == "test":
        history = list(test_data)
        n_pred = len(history)
    elif mode == "forecast":
        n_pred = n_pred
        history = list(train_data + test_data)

    for t in range(n_pred):
        model = sm.tsa.ARIMA(history, order=(self.p, self.d, self.q))
        model_fit = model.fit()
        output = model_fit.forecast()
        prediction = output[0]
        predictions.append(prediction)

        if mode == "test":
            history.append(test_data[t])
        elif mode == "forecast":
            history.append(prediction)

    return predictions
```

4.10. ábra. A tesztadatok becslése előre-lepő validációval, illetve előrejelzések készítése (ARIMA).
Forrás: saját szerkesztés Brownlee (2023) alapján.

A 4.6. táblázatban megfigyelhető, hogy Kovászna megyénél az ARIMA (2, 1, 2) modell hiába illeszkedett hasonló pontossággal a tanító adatokra, mint a többi két idősor modellje (4.5. táblázat), jelentősen gyengébben általánosított, mint Hargita és Maros megye ARIMA modelljei. Ez véleményem szerint többek között azért volt így, mivel a többi két megyéhez képest Kovászna megyében lényegesebben radikálisabb változások jellemzik az idősort.

Például 2023 júliusa és októbere között 1.4%-os emelkedés történt (4.9. ábra), míg a másik két megyében minimális volt a változás ezen időszak alatt. Ettől eltekintve a hibamutatók értékei viszonylag így is alacsonynak számítanak.

Megye	Modell	MSE	RRMSE	MAPE	R ²
Kovászna	ARIMA (2, 1, 2)	0.12	0.08	7.15	0.72
Hargita	ARIMA (2, 1, 2)	0.03	0.04	3.51	0.64
Maros	ARIMA (1, 1, 0)	0.04	0.07	4.29	0.35

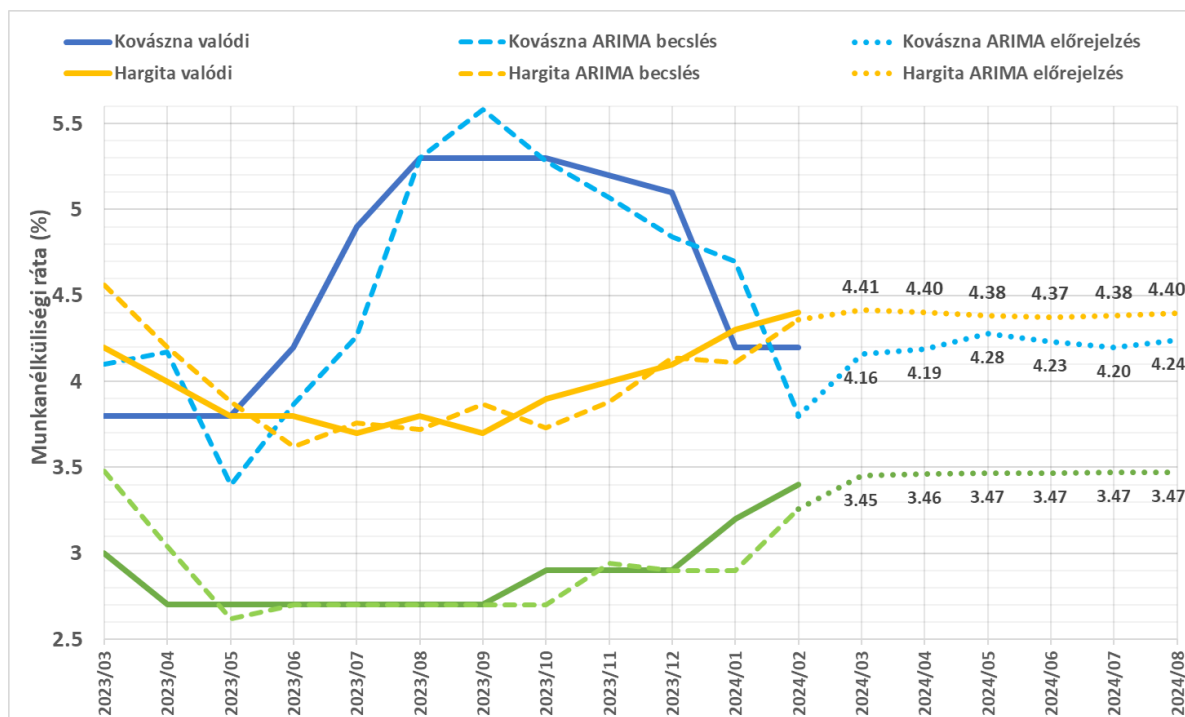
4.6. táblázat. Az ARIMA modellek teljesítménye a teszt adatbázison (saját szerkesztés)

A 4.11. ábrán szemléltetem mindhárom megye valódi adatait 2023 március és 2024 február között (folytonos vonalak), valamint az ARIMA modellek által adott becsléseket (szaggatott vonalak) és az előrejelzéseket (pontosított vonal).

4.3.3. Előrejelzések az ARIMA modellekkel

A kért előrejelzések számát a modell paraméterezése során adhatjuk meg (4.8. ábra). Miután a modell illesztésre került, és kiértékelődött a tesztadatokra adott becslések pontossága, létrejönnek az előrejelzések. Az előrejelzéseket Brownlee (2023) alapján, a becslések készítéséhez hasonló módon, ugyancsak a 4.10-es ábrán levő függvénnyel számítom ki.

Azonban ilyenkor a modell már a teljes ismert idősorra illeszkedik, majd egy-egy értéket jelez előre. Minden lépésben, az előrejelzett értékkel bővíti az idősort, és azzal újra illeszti önmagát. Ezzel a módszerrel az előrejelzések a lehető legpontosabbak lesznek, mivel a modell az összes rendelkezésre álló adatot felhasználja. Az eredmények összehasonlíthatósága érdekében grafikonokon ábrázoltam a tesztadatokat, az azokra adott becsléseket és az előrejelzéseket is (4.11. ábra).



4.11. ábra. Az ARIMA modellek becslései és előrejelzései (saját ábra).

A 2024 márciusától kezdődő előrejelzések azt mutatják, hogy a Hargita, Kovászna és Maros megye munkanélküliségi rátái várhatóan enyhén emelkedni fognak a nyár végéig.

Azonban nyáron csökkennie kellene a munkanélküliségnek (4.3. ábra), így ezt nem igazán tudta megtanulni egyik ARIMA modell sem. Ebben a szezonális ARIMA (SARIMA) modell tudna segíteni, ami viszont jóval bonyolultabb és számításigényesebb modell. Emellett fontos megjegyezni, hogy az ARIMA modellek általában csak rövid- vagy középtávon teljesítenek jól.

4.4. A neuronhálók implementációja

Ebben a részben azt fejtém ki, hogy milyen módszerrel valósítottam meg a neurális hálózatokat, hogyan tanítottam be őket, és hogyan jeleztem előre velük.

4.4.1. A tanító és tesztelő adatbázis előkészítése

Egy idősor megfigyeléseit át lehet alakítani felügyelt tanulási problémaként. Ezt úgy tehetjük meg, hogy az előző időpontot vagy időpontokat bemeneti változóként, és a következő időpontot kimeneti változóként kezeljük.

Az idősorokból Brownlee (2018), 44. oldal alapján készítettem el a felügyelt tanításhoz szükséges tanító- és tesztelő párokat (x_{train} , y_{train} , x_{test} , y_{test}), a szakirodalomban sliding window néven ismert módszerrel. Ilyenkor az idősorból egy választott ablakméret alapján hozzuk létre a bemenetek és kimenetek listáját. Egy kimeneti értéket a választott ablakméretnek megfelelő számú korábbi megfigyelésből állítunk elő, vagyis egy kimenetet n darab korábbi értékből

következik.

```
def split_sequence(sequence, n_steps):
    x, y = list(), list()
    for i in range(len(sequence)):
        end_ix = i + n_steps
        if end_ix > len(sequence)-1:
            break
        seq_x, seq_y = sequence[i:end_ix], sequence[end_ix]
        x.append(seq_x)
        y.append(seq_y)
    return np.array(x), np.array(y)
```

4.12. ábra. Az idősorokból tanító párokat előállító Python függvény.

Forrás: Brownlee, (2018), 47. oldal alapján

A bemeneti minták hosszának (window size) 3-at választottam, mert a szakirodalomban ez a legelterjedtebb, például (Brownlee, 2018) is ezt az ablakméretet használta. A 4.13. ábrán reprezentálom, hogy Maros megye idősorának első 5 megfigyeléséből hogyan fog kinézni az első kettő tanító pár.

```
time_series = [8.3, 8.4, 8.5, 8.2, 7.9]
lag = 3
X_train[0] = [8.3, 8.4, 8.5]      Y_train[0] = 8.2
X_train[1] = [8.4, 8.5, 8.2]      Y_train[1] = 7.9
```

4.13. ábra. Példa a mozgóablak módszerrel készített tanító párokra (saját szerkesztés).

Ez a gyakorlatban azt jelenti, hogy például a januári, februári és márciusi megfigyelésekből következik az áprilisi érték. A májusi megfigyelés pedig a februári, márciusi és áprilisi értékektől függ, és így tovább. Ez a program szempontjából azt jelenti, a bemeneti listák (x_{test} , x_{train}) kétdimenziósak lesznek, míg a targetek (y_{test} , y_{train}) egydimenziósak lesznek.

4.4.2. Az adatok előfeldolgozása

A neuronhálós modellek érzékenyek lehetnek az adatok eloszlására és azok értéktartományára. A kiugró értékek lelassíthatják vagy rossz irányba vihetik el a tanulást, mivel a magasabb értékek dominálhatják a célfüggvényt. (Brassai, 2019).

Emiatt ajánlott az adatok skálázása, mielőtt azokkal betanítanánk egy mesterséges neuronhálót. Az adatok skálázása biztosítja, hogy minden adatnak ugyanaz legyen az értéktartománya, ami hozzájárulhat a neuronháló hatékonyabb tanításához azáltal, hogy segít kiküszöbölni a lokális minimumokba való ragadást (Brassai, 2019).

A **standardizálás** megpróbálja az adatok átlagát nullára és szórását egyre állítani $N(0,1)$. Ilyenkor a megfigyelésekből (x_i) kivonjuk az átlagot ($\bar{x}$), majd elosztjuk a szórással (σ) (Brassai, 2019).

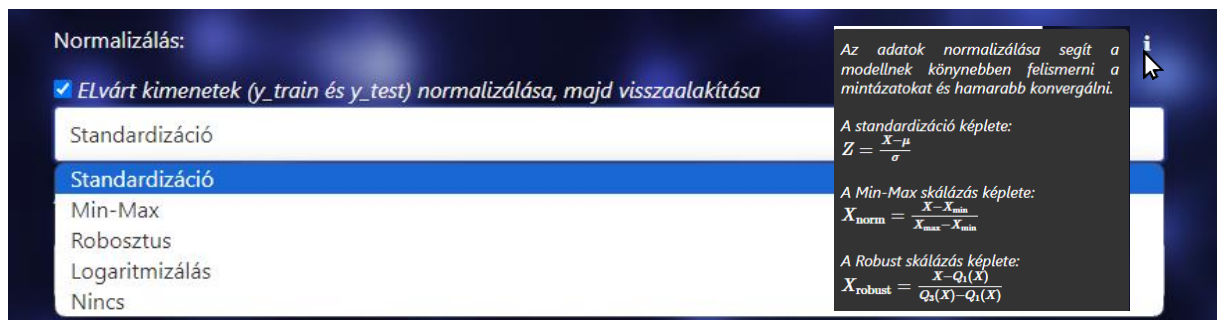
$$x' = \frac{x_i - \bar{x}}{\sigma} \quad (4.1)$$

A programomban a `sklearn.preprocessing` osztály segítségével többféle transzformációra is lehetőséget biztosítok az MLP neurális hálók készítése során, Brownlee (2019) alapján: egy legördülő listában választhatunk standardizáció, min-max normalizáció, robosztus skálázás és logaritmizálás között (4.14. ábra).

Az is fontos kérdés, hogy csak a bemeneteket (x - input), vagy az elvárt kimeneteket (y - target) is transzformáljuk-e? Mindkét megközelítés elfogadható, különböző idősorok és modellek esetében máshogy hat a tanulási teljesítményre.

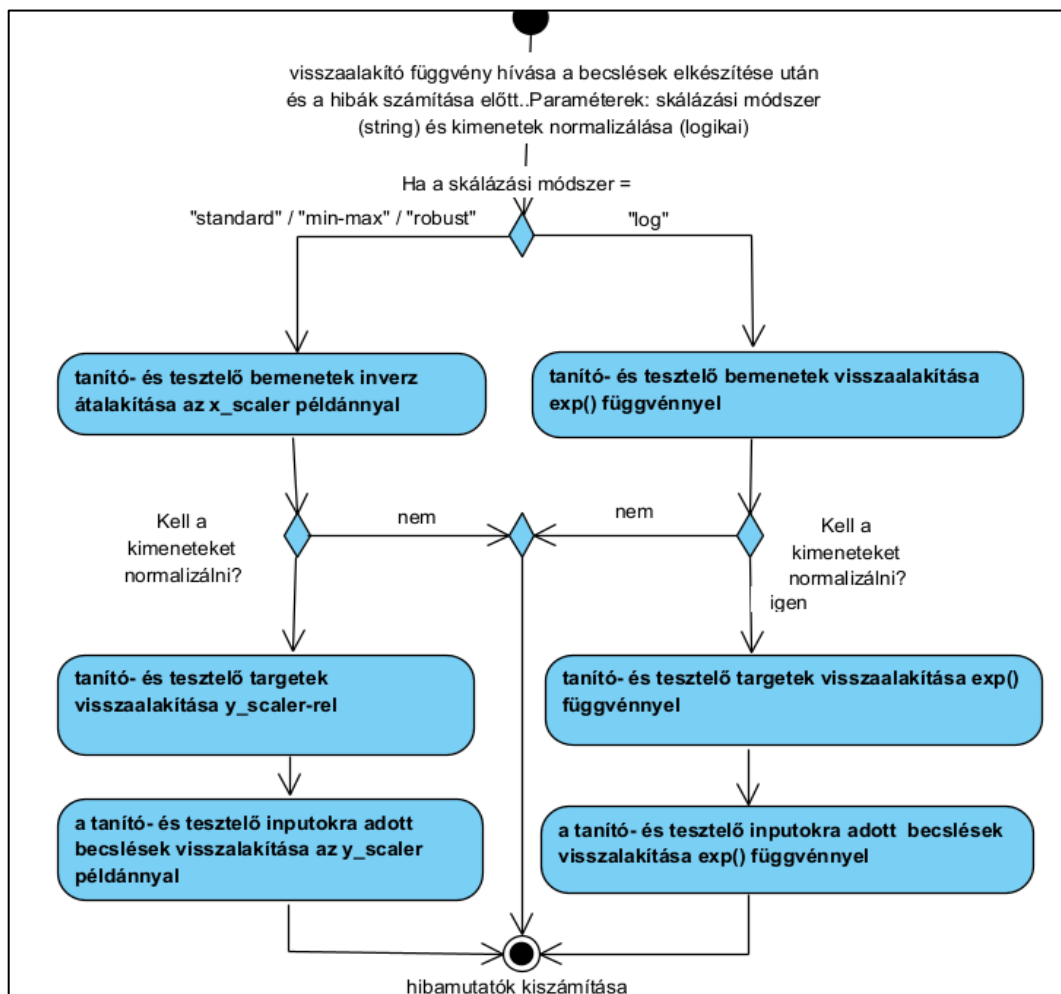
A kutatás során kipróbáltam többféle skálázást a felsoroltak közül, és mindegyik megye esetében az idősorok standardizálása után teljesítettek a legjobban az MLP regressziós modellek (az összes érték, tehát a bemenetek és a kimenetek is át lett alakítva). A modell által számított értékek visszaállítására lehetőség van a skálázó példány `inverse_transform()` függvénye hívása révén, illetve a logaritmizált értékek eredeti skálán belüli értékét az

exponenciális függvénnyel számolja ki a program. Az adatok skálázásának, a becslések készítésének és azok visszaalakításának logikája a 4.15. ábrán látható.



4.14. ábra. Az adatok előfeldolgozását beállító űrlap.

Valamennyi beállítható neuronháló paraméterhez tartozik egy információs ikon, amely fölé az egérkurzort helyezve, hasznos információkat kapunk egy ablakban. Ez segít a felhasználónak abban, hogy tudatosan állítsa be az egyes paramétereket.

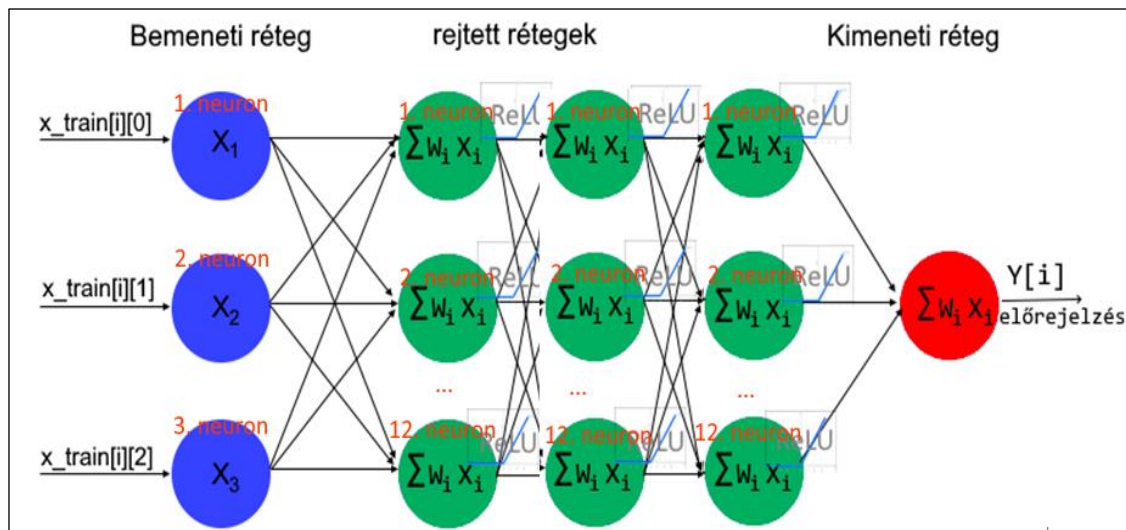


4.15. ábra. Az MLP regressziós modell által becsült értékek visszaalakításának logikája.

Forrás: saját szerkesztés Visual Paradigm programban

4.4.3. Az MLP modellek implementációja

(Heaton, 2005) alapján és próbálgatások után választottam ki a megfelelő hálószerkezetet: mivel mozgó ablak mérete 3 volt (3 korábbi adatból következik egy negyedik), ebből adódóan mindegyik MLP modellnél 3 bemeneti neuront használtam. Mivel egy lépésben egy darab előrejelzést ad a modell a bemeneti mintából, a kimeneti rétegben egyetlen neuron van. Az MLP modellbe végül három rejtett réteget, bennük 12 neuront tettem, így elérve a legjobb általánosító képességet.



4.16. ábra. Az előrejelzésekhez használt MLP hálók szerkezete.

Forrás: Saját szerkesztés (Brassai, 2019), 80. oldal alapján

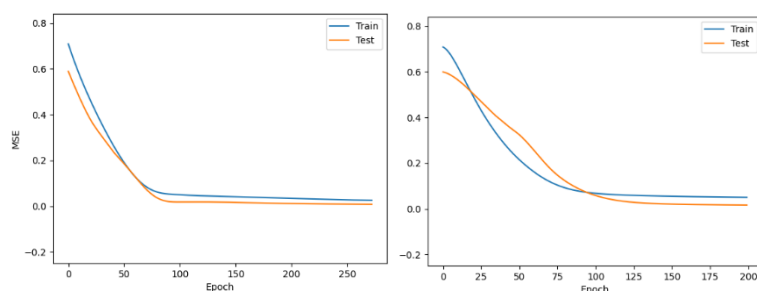
A legjobb MLP modell keresése során összehasonlítottam, hogy az egyes idősorok esetében melyik optimalizációs algoritmus (ADAM/LBFGS/SGD), illetve az adatok milyen transzformációja (standardizálás, min-max normalizáció, robosztus skálázás, logaritmizálás, vagy egyik sem) mellett illeszkedtek, illetve általánosítottak legjobban a modellek. Mindhárom idősor esetében ugyanaz a hálószerkezet és hiperparaméterek bizonyultak a legjobbnak.

Az idősorok standardizálása (bemenetek és elvárt kimeneteket egyaránt) segített a legjobban a modellnek a tanulásban, az LBFGS optimalizációs algoritmus által rövidebb idő alatt konvergáltak a modellek, és ezen állapotban jóval kisebb MSE/RRMSE/MAPE mutatókat értek el a teszt adatokon, mint az ADAM vagy SGD esetében. Emellett a reziduumok eloszlása, autokorrelációja is jobb volt az LBFGS használatával. Az L2 regularizációs együttható az alapértelmezett, 0.0001 érték volt.

4.4.4. A modellek tanítása és tesztelése

A neuronháló példányok létrehozását követően az MLPRegressor könyvtár `model_fit` függvényével automatikusan lehet tanítani a modelleket, ami figyeli a veszteségeket, így le tud állni, ha pl. 20 tanítási korszak után sem javul a veszteség (early stopping). A `partial_fit` függvénnyel pedig iteratíván, lépésenként (korszakonként) lehet tanítani a modellt.

A teszt adatbázisokon való veszteségeket manuálisan számítottam ki iteratíván, minden lefuttatott epoch után. Ezekből hozom létre a veszteség görbét (4.17. ábra).



4.17. ábra. A program által készített veszteség görbék (saját ábra).

Az eredmények reprodukálhatóságának érdekében egy véletlen súlyozási kezdőértéket lehet megadni a modellnek a létrehozás során. Különböző kezdőértékek és állandó egyéb hiperparaméterek mellett azonban eltérő általánosító képességet ér el a modell. Emiatt lehetőséget adok 1 és 100 közötti kezdőértékek közül a legjobbat megkeresni.

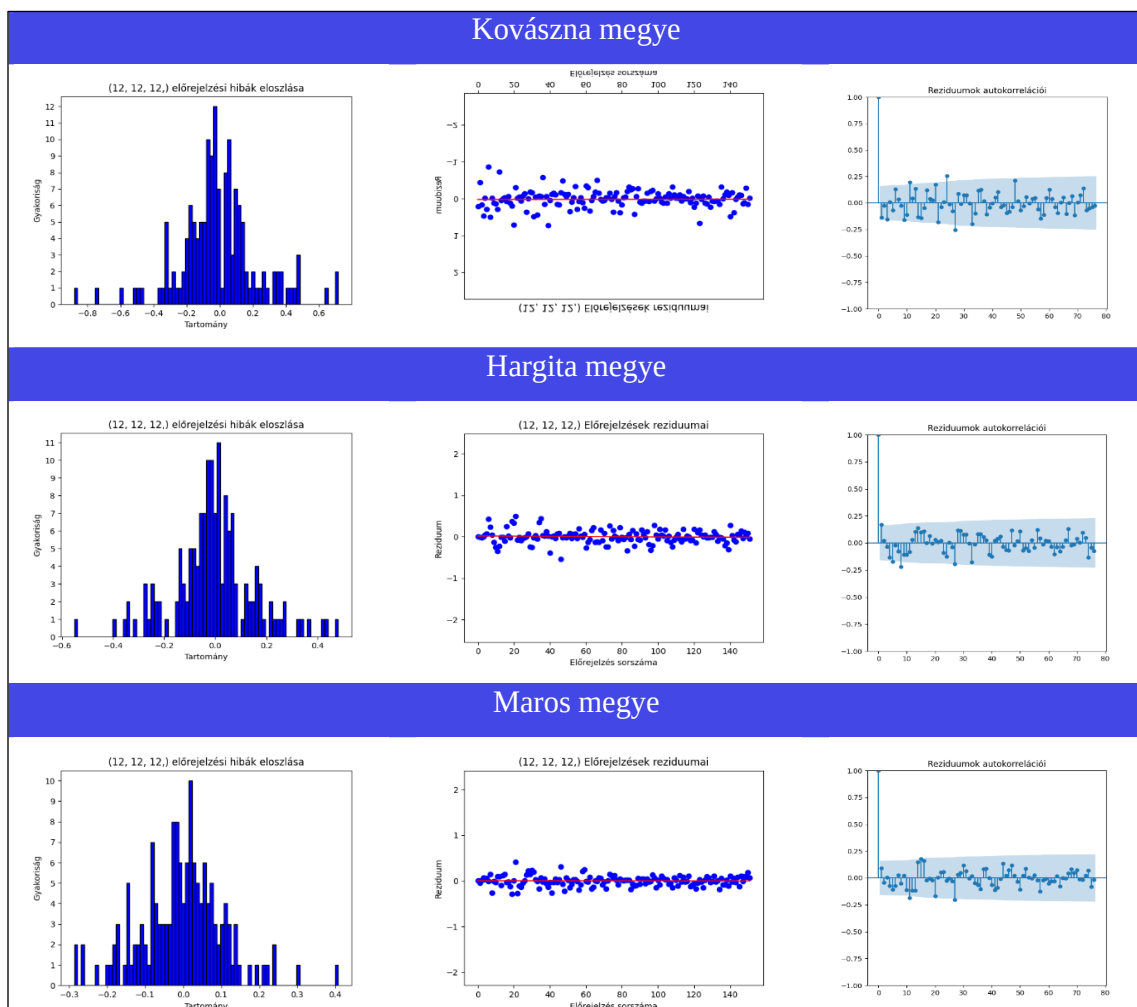
Minden egyes kezdőértékkel betanítom a modellt, majd lekérem a tesztadatokra a becsléseket és kiértékelem azokat. A legkisebb, vagy a megadott küszöbértéket elérő MSE értékkel általánosító modellt tartja meg a program.

A legjobban általánosító MLP regressziós modellek illeszkedését a 4.7-es táblázatban levő hibamutatókkal szemléltetem.

Megye	Modell	MSE	RRMSE	MAPE	R ²
Kovászna	MLP (12, 12, 12)	0.06	0.04	3.01	0.99
Hargita	MLP (12, 12, 12)	0.03	0.03	2.22	0.99
Maros	MLP (12, 12, 12)	0.01	0.02	1.99	0.99

4.7. táblázat. Az MLP modellek illeszkedésének mutatói (saját szerkesztés).

A 4.7-es táblázatban látható, hogy a neuronhálók elég pontosan megtanulták a tanító bemenetekre adott helyes válaszokat, sokkal kisebbek a hibamutatók, mint az ARIMA (4.4. táblázat). esetében, valamint a reziduumok követik a normális eloszlást, varianciájuk állandó és elhanyagolhatóak az autokorrelációs kapcsolatok (4.18. ábra).



4.18. ábra. Az MLP modellek tanulásból származó reziduumok.

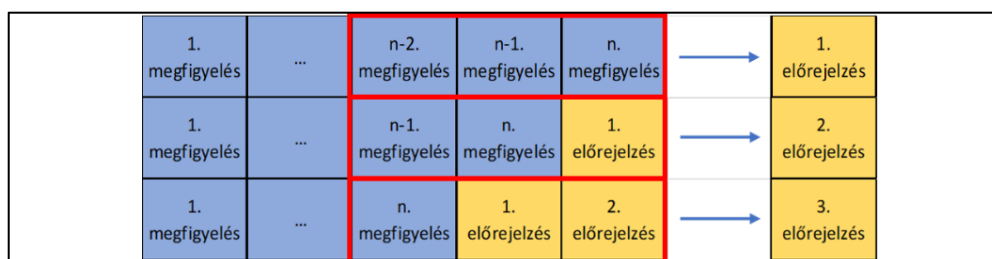
A következő lépésben, a betanított hálókától lekértem a teszt bemenetekre (x_{test}) adott becsléseket, és kiszámoltam az elvárt kimenetekhez (y_{test}) viszonyítva a hibákat. A hiba statisztikák a 4.8-as táblázatban láthatóak.

Megye	Modell	MSE	RRMSE	MAPE	R^2
Kovászna	MLP (12, 12, 12)	0.06	0.06	4.18	0.83
Hargita	MLP (12, 12, 12)	0.01	0.02	1.92	0.85
Maros	MLP (12, 12, 12)	0.02	0.05	3.14	0.61

4.8. táblázat. Az MLP modellek általánosítási képességei (saját szerkesztés).

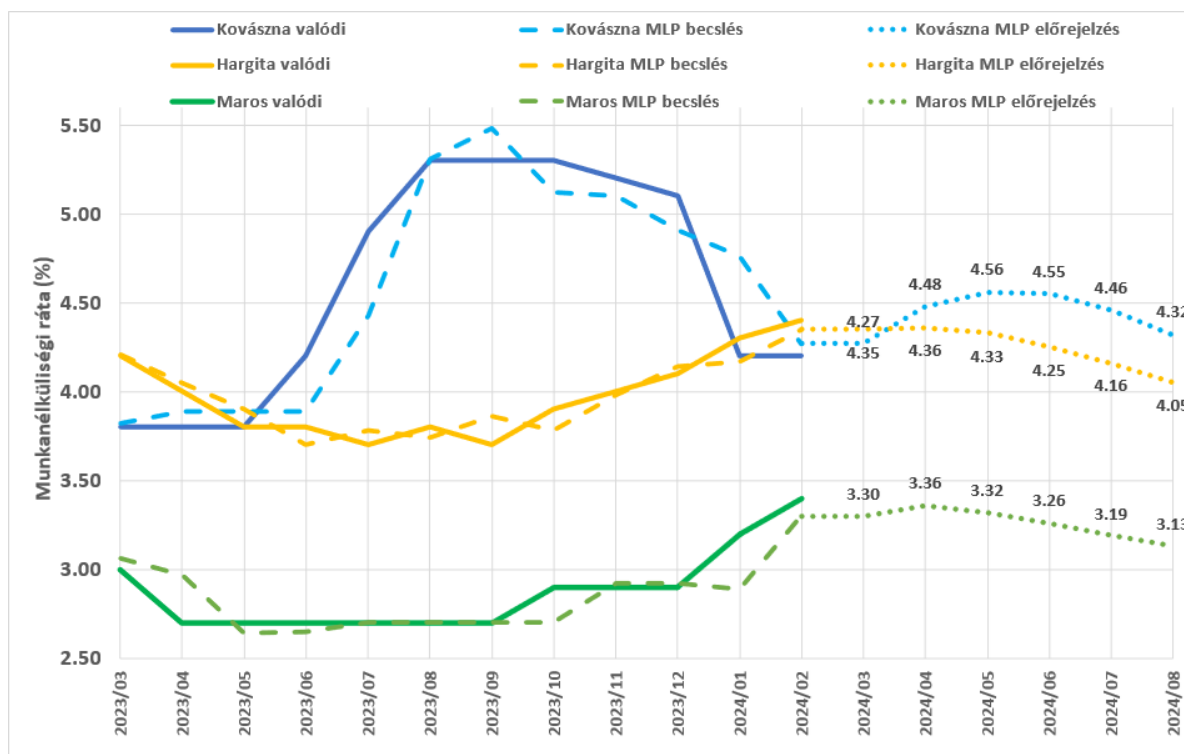
4.4.5. Rekurzív előrejelzés az MLP modellekkel

Az MLP regressziós modelleknél Brownlee (2018) alapján úgy jártam el, hogy a modellnek megtanítottam a teljes idősort, majd a tesztadatok legutolsó bemeneti mintája alapján először egy előrejelzést kértem le. Ezután az előrejelzett értékkel frissítettem a bemeneti mintát úgy, hogy eggyel balra toltam az értékeket, így bemeneti minta legelső (legrégebbi) eleme kiesik, és a minta végére az előrejelzett érték jön (4.19-es ábra). A rekurzív előrejelzéseket készítő függvény az 1. mellékletben található.



4.19. ábra. Rekurzív előrejelzés hármasképp (saját szerkesztés).

Az elkészült előrejelzések a 4.20. ábrán láthatóak. A neuronhálók megtanulták, hogy nyáron általában csökkenni szokott a munkanélküliség az előző évszakokhoz képest. Emiatt az MLP regressziós modellek előrejelzéseit megbízhatóbbnak mondanám, mint az ARIMA modellekkel készített becsléseket (4.11-es ábra).



4.20. ábra. Az MLP regressziós modellek becslései és előrejelzései.

5. Eredmények

Ebben a részben röviden összefoglalom azt, hogy hogyan általánosítottak az ARIMA és az MLP regressziós modellek. Maros megyében az ARIMA (1, 1, 0) modell hasonló teljesítményt nyújt, mint az MLP modell, de az R² értéke még itt is alacsonyabb. Bár néhány esetben az ARIMA modell is elfogadható jó teljesítményt mutat, általában alulmarad az MLP modellekhez képest. Különösen Hargita megyében kiemelkedő az MLP modell, ahol a legmagasabb R² értéket és a legalacsonyabb hibamértékeket érte el a teszt adatbázison.

Megye	Modell	MSE	RRMSE	MAPE	R ²
Kovászna	MLP (12, 12, 12)	0.06	0.06	4.18	0.83
Kovászna	ARIMA (2, 1, 2)	0.15	0.08	5.52	0.65
Hargita	MLP (12, 12, 12)	0.01	0.02	1.92	0.85
Hargita	ARIMA (2, 1, 2)	0.02	0.03	2.88	0.58
Maros	MLP (12, 12, 12)	0.02	0.05	3.14	0.61
Maros	ARIMA (1, 1, 0)	0.02	0.05	3.22	0.45

4.9. táblázat. Az MLP és ARIMA modellek hiba tesztjeinek összefoglalása.

Az MLP modellek, a háromrétegű (12, 12, 12) hálószerkezettel, ReLU aktivációs függvényel és LBFGS optimalizációs algoritmussal jobb eredményeket produkáltak mindhárom megye esetében, mint az ARIMA modellek. Ez arra utal, hogy az MLP modellek jobban képesek megragadni a bonyolult mintázatokat és nemlineáris összefüggéseket az adatokban, mint az ARIMA modellek.

Az előrejelzési adatok alapján az MLP modellek nagyobb változékonyságot mutatnak, míg az ARIMA modellek inkább stabilabb előrejelzéseket produkáltak.

A modellek által adott becslések és előrejelzések értékeit az 5. mellékletben levő táblázatban foglaltam össze. Emellett a 6. és 7. mellékletben levő ábrákon a tesztelési hibákat is feltüntettem (eloszlás, autokorrelációk). A teszt reziduumok az ARIMA és az MLP modellek esetében is követik a normális eloszlást, és nincs közöttük autokorreláció, valamint varianciájuk állandó, tehát megbízhatóan teljesítettek a modellek.

5. Következtetések

A kutatás során Hargita, Kovászna és Maros megye munkanélküliségi rátáit modelleztem ARIMA, és MLP regressziós modellekkel. Minden számítást Python-ben végeztem. Tanító adatbázisnak a 2010 január és 2023 február közötti havi adatokat használtam fel és elemeztem. A modellek általánosító képességének vizsgálatához a 2023 március és 2024 február megfigyeléseket használtam.

A Box-Jenkins eljárással kiválasztott ARIMA (2, 1, 2) és ARIMA (1, 1, 0) modellek RRMSE értékei a tesztelés során 0.03 és 0.08 között alakultak, míg a rácskereséssel hiperparaméterezett MLP regressziós modelleké 0.02 és 0.06 között.

Mindhárom megye idősorára az MLP (12, 12, 12) hálószerkezetet használtam, ReLU aktivációs függvényel, LBFGS tanulási algoritmussal.

Az ARIMA előrejelzések szerint stagnálás, míg az MLP modellek szerint a nyár elejéig emelkedés, majd a nyár végéig fokozatos csökkenés várható.

Eredményeink összhangban vannak Ajoodha és Mulaudzi (2020), Tufaner és Sözen (2021) eredményeivel, abból a szempontból, hogy az MLP neurális hálók átlagosan jobban teljesítettek a munkanélküliségi ráták előrejelzésében (mindhárom idősor esetén), mint az ARIMA modellek. Sasu (2013) kutatásának eredménye is összhangban van azzal, hogy az MLP modellek pontosabban jelezték előre a munkanélküliségi rátákat, mint az ARIMA modellek.

Paszkievicz és társai (2023) eredményei szerint, a regressziós problémában az LBFGS optimalizációs algoritmus mellett pontosabb becsléseket adott az MLP modell, mint az ADAM használata esetén. Pardede és társai (2022) eredményei alapján a lineáris klasszifikáció terén jobban teljesített az LBFGS algoritmus, mint az ADAM vagy SGD. A kutatásom során azt tapasztaltam, hogy a többrétegű perceptron prediktív modellek jobban általánosítottak az LBFGS, mint az ADAM vagy az SGD használata mellett.

Azt a következtetést vontam le, hogy a gépi tanulás rendkívül jó alternatívát jelenthet a Box-Jenkins eljárásra, érdemes további vizsgálatokat végezni, más idősorokkal, hogy megállapíthassuk a módszerben rejlő lehetőségeket és korlátokat.

A Django keretrendszer, a Python alprogramokkal összekötve lehetővé teszi, hogy a prediktív neuronháló modellek, és velük az előrejelzések készítése felhasználóbarát legyen. A programom nehezen elérhető Python elemzési modulokat, akárcsak az adat vizualizációt, könnyen elérhetővé teszi, akárcsak bármelyik mai korszerű statisztikai program. A programot könnyen fel lehet használni hasonló kutatásokra, más idősorokra is. Ezenkívül bárki felhasználhatja a saját idősorainak előrejelzésére.

6. Irodalomjegyzék

- Adenomon, M. O. (2017). Modelling and forecasting unemployment rates in Nigeria using ARIMA model. *FUW Trends in Science and Technology Journal*, 2(1B), 525-531.
- Ajoodha, R., & Mulaudzi, R. (2020. November 25-27). An Exploration of Machine Learning Models to Forecast the Unemployment Rate of South Africa: A Univariate Approach. *An Exploration of Machine Learning Models to Forecast the Unemployment Rate of South Africa: A Univariate Approach*. Kimberley, Dél-Afrika: IEEE. doi:10.1109/IMITEC50163.2020.9334090
- Bamberger, S., Heckel, R., & Krahmer, F. (2023. 08 5). *Approximating Positive Homogeneous Functions with Scale Invariant Neural Networks*. doi:10.48550/arXiv.2308.02836
- Brassai, S. T. (2019). *Neurális hálózatok és Fuzzy logika*. Kolozsvár: Scientia Kiadó. Forrás: <http://real.mtak.hu/id/eprint/122603>
- Brownlee, J. (2018). *Predict the Future with MLPs, CNNs and LSTMs in Python*. Forrás: <https://dokumen.pub/deep-learning-for-time-series-forecasting-predict-the-future-with-mlps-cnns-and-lstms-in-python.html>
- Brownlee, J. (2019). How to Scale Data for Long Short-Term Memory Networks in Python. Forrás: <https://machinelearningmastery.com/how-to-scale-data-for-long-short-term-memory-networks-in-python/>
- Brownlee, J. (2023. november 18). *How to Create an ARIMA Model for Time Series Forecasting in Python*. Letöltés dátuma: 2024. március, forrás: machinelearningmastery.com: <https://machinelearningmastery.com/arima-for-time-series-forecasting-with-python/>
- Davidescu, A. A., Apostu, S.-A., & Paul, A. (2021). Comparative Analysis of Different Univariate Forecasting Methods in Modelling and Predicting the Romanian Unemployment Rate for the Period 2021–2022. *Entropy*, 23(325), 324. doi:10.3390/e23030325
- Derbentsev, V., Matviychuk, A., Datsenko, N., Bezkorovainyi, V., & Azaryan, A. (2020). Machine learning approaches for financial time series. *M3E2-MLPEED 2020* (old.: 434-450). Odessa: CEUR Workshop Proceedings. doi:<https://doi.org/10.31812/123456789/4478>
- Dhandi, A. (2024). *Mastering Django: Projects for Beginners*.

- Didiharyono, D., & Syukri, M. (2020. 03 03). Forecasting with ARIMA Model in Anticipating Open Unemployment rates in South Sulawesi. *International journal of scientific and technology research*, 9, 3838-3841. Forrás: <https://osf.io/f5zh6/>
- Drelczuk, K. (2020. 05 07). *ACF (autocorrelation function) — simple explanation with Python example*. Forrás: Medium: <https://medium.com/@krzysztofrelczuk/acf-autocorrelation-function-simple-explanation-with-python-example-492484c32711>
- Georgeta, E. S. (2015). *The economic and social situation in Romania*. Brussel, Belgium: European Economic and Social Committee. doi:10.2864/484519
- Heaton, J. (2005). *Introduction to Neural Networks with Java*. St. Louis: Heaton Research.
- Holbrook, R., & Cook, A. (2023). *Overfitting and Underfitting*. Forrás: Kaggle.com: <https://www.kaggle.com/code/ryanholbrook/overfitting-and-underfitting>
- INSSE. (2016). Metodologie Somaj BIM lunar. Forrás: https://insse.ro/cms/files/statistici/comunicate/somaj%20BIM/a16/precizari_metodologie_ian2016.pdf
- Lázár, E. (2022). *Közgazdasági kutatómódszertan*. Kolozsvár: Risoprint kiadó.
- Madaras, S. (2014). A gazdasági válság hatása a munkanélküliség alakulására országos és megyei szinten Romániában. *Közgazdász Fórum*, 17 (1-2), 136–149. Forrás: <https://epa.oszk.hu/00300/00315/00108/pdf/>
- Madaras, S. (2018). Forecasting the regional unemployment rate based on the Box-Jenkins methodology vs. the Artificial Neural Network approach. Case study of Braşov and Harghita counties. *Közgazdász fórum*, 21(135), 66-79. Letöltés dátuma: 2024. február
- Madaras, S. (2024). Deep Learning Algorithm Forecasting the Unemployment Rates in the Central European Countries. *Economics and Business*, 86-102. doi:10.7250/eb-2024-0006
- Madaras, S., & Györfy, L. (2016. december). Non-Linearity and Non-Stationarity of Exchange Rate Time Series in Three Central-Eastern European Countries Regarding the CHF Currency in 2014 and 2015. *Acta Universitatis Sapientiae Economics and Business*, 4(1), 33-41. doi:10.1515/auseb-2016-0002
- Oukhouya, H., & El Himdi, K. (2023). Comparing Machine Learning Methods—SVR, XGBoost, LSTM, and MLP— For Forecasting the Moroccan Stock Market. *Computer Sciences & Mathematics Forum*, 7(39). doi:10.3390/IOCMA2023-14409
- Pardede, D., & Firmansyah, I. e. (2022). COMPARISON OF MULTILAYERPERCEPTRON'SACTIVATION AND OP-TIMIZATION

FUNCTIONS IN CLASSIFICATION OF COVID-19 PATIENTS. *Jurnal Teknologi dan Sistem Informasi*, 271-278.

- Ramli, S. F., Fidaus, M., Uzair, H., Khairi, M., & Zharif, A. (2018). Prediction of the Unemployment Rate in Malaysia. *International Journal of Modern Trends*, 1(4), 38-44. Forrás: <http://www.ijmtss.com/PDF/IJMTSS-2018-04-12-04.pdf>
- Sándor, Z. (2019). *Bevezetés az ökonometriába*. Sepsiszentgyörgy: T3 Kiadó.
- Sasu, A. (2013). A quantitative comparison of models for univariate time series forecasting. *Bulletin of the Transilvania University of Brasov. Series III: Mathematics and Computer Science*, 6(55), 117-124. Forrás: https://webbut.unitbv.ro/index.php/Series_III/article/view/2005
- Tóth, A., Kálmán, B., & Juhász, T. (2021). Unemployment development in the Visegrád Group countries as a result of the COVID-19 pandemic. *Reproduction of Human Capital – Mutual Links and Connections (RELIK 2021)*, 780-787. Forrás: <https://real.mtak.hu/id/eprint/191199>
- Tufaner, M. B., & Sözen, İ. (2021). Forecasting Unemployment Rate in the Aftermath of the Covid-19 Pandemic: The Turkish Case. *İzmir Journal of Economics*, 36, 685 - 693. doi:10.24988/ije.202136312
- Yamacli, D. S., & Yamacli, S. (2023). Estimation of the unemployment rate in Turkey: A comparison of the ARIMA and machine learning models including Covid-19 pandemic periods. *Helyion*, 9(1). doi:10.1016/j.heliyon.2023.e12796

7. Mellékletek

```
def recursiveForecast(self, n, x_full, y_full, scaler_y=None, scalerMode="-", normOut = False):
    self.model.fit(x_full, y_full)
    forecasts = []
    input = x_full[-1].reshape(1, -1)

    for i in range(n):
        forecast = self.predict(input)[0]
        if(normOut == False):
            if(scalerMode == "log"):
                forecast = np.log(np.longdouble(forecast))
            forecasts.append(forecast)
            input = np.hstack((input[:, 1:], forecast.reshape(1, -1)))
        if scaler_y is not None and not normOut:
            input[0][-1] = scaler_y.transform(np.array(input[0][-1]).reshape(-1, 1))
    if(scalerMode == "log"):
        forecasts = np.exp(forecasts).tolist()
    elif(scaler_y is not None and normOut):
        forecasts = scaler_y.inverse_transform(np.array(forecasts).reshape(1, -1)).flatten().tolist()

    return forecasts
```

1. melléklet. Rekurzív előrejelzéseket készítése MLP regressziós modellel.

Forrás: saját szerkesztés, (Brownlee, 2018) alapján.

```
def train_model(self, x_train: list, y_train: list, x_test: list, y_test: list, n: int, earlyStop: bool, solver: str):
    """If solver is ADAM or SGD, it fits the model step-by-step and calculates loss curves.
    In case of LBFGS it uses the autamitc fit() function and the curves won't be calculated since its not possible"""
    self.x_train = x_train
    self.y_train = y_train
    self.x_test = x_test
    self.y_test = y_test

    self.trainLossCurve = []
    self.testLossCurve = []

    if(str.upper(solver) == "ADAM" or str.upper(solver) == "SGD"):
        n = max(n, 200)
        if(not earlyStop):
            n = self.max_iters

    for i in range(int(n)):
        self.model = self.model.partial_fit(x_train, y_train)
        predictions = self.model.predict(x_train)
        train_loss = MSE(predictions, y_train)
        self.trainLossCurve.append(train_loss)

        predictions = self.model.predict(self.x_test)
        test_loss = MSE(predictions, self.y_test)
        self.testLossCurve.append(test_loss)

    else:
        self.model = self.model.fit(x_train, y_train)

    self.weights = [layer_weights for layer_weights in self.model.coefs_]
    learning_pred = self.model.predict(x_train)
    self.learning_pred = learning_pred
```

2. melléklet. Az MLP modellek illesztése.

Forrás: saját szerkesztés, (Brownlee, 2018) alapján.

Rejtett rétegek neuronjainak száma (pl. 12,12,12):

Maximum iterációk (epoch) száma: i

Korai leállítás: ☒ i

Random állapot kezdőérték: i

Random állapot utolsó érték:

Cél MSE (%): i

Normalizálás:

☒ *Elvárt kimenetek (y_train és y_test) normalizálása, majd visszaalakítása*

Aktivációs függvény:

Optimalizációs Algoritmus:

Tanulási ráta (csak SGD esetében):

Impact of Regularization

Az L2 (Ridge regresszió) hozzáadja a veszteséghez a súlyok négyzetösszegét egy λ együtthatóval szorozva.

$$Loss = Error(y, \hat{y}) + \lambda \sum_{i=1}^N w_i^2$$

Túl kicsi λ : túltanítás veszélye

Túl nagy λ : alultanulás veszélye

L2 regularizációs együttható (λ): i

Tanító adatbázis készítése: késleltetett értékek (ablakméret): i

Előrejelzések száma a tesztadatokon túl: i

MLP előrejelzések készítése

3. melléklet. Az MLP regressziós modellek paraméterezését szolgáló űrlap.

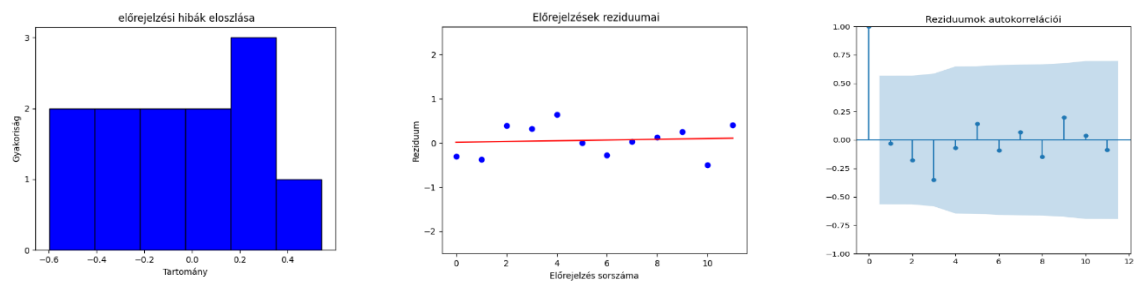
	Eredeti idősor				Első differencia			
idősor	ADF teszt		KPSS teszt		ADF teszt		KPSS teszt	
	Statisztika	p-érték	Statisztika	p-érték	Statisztika	p-érték	Statisztika	p-érték
	Kritikus érték (5% szig. szint): (-2.88)		Kritikus érték (5% szig. szint): (0.46)		Kritikus érték (5% szig. szint): (-2.88)		Kritikus érték (5% szig. szint): (0.46)	
Kovászna	-2.73	0.07	1.57	0.01	-3.44	0.01	0.44	0.06
Hargita	-2.7	0.07	1.5	0.01	-2.79	0.06	0.22	0.1
Maros	-2.51	0.11	1.69	0.01	-6.63	0.0	0.12	0.1
nem stacionárius nem stacionárius (de közel a határértékhez) stacionárius								

4. melléklet. Példa a program által készített, a stacionaritást elemző táblázatra.

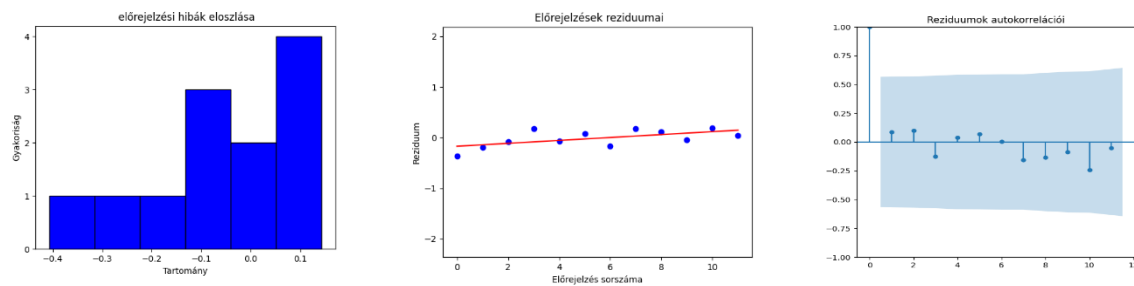
	megye	Kovászna			Hargita			Maros		
	modell	valódi adat	ARIMA (2, 1, 2)	MLP	valódi adat	ARIMA (2, 1, 2)	MLP	valódi adat	ARIMA (1, 1, 0)	MLP
	dárum	%								
teszt időszak	2023/03	3.80	4.10	3.82	4.20	4.56	4.21	3.00	3.48	3.06
	2023/04	3.80	4.17	3.89	4.00	4.20	4.05	2.70	3.04	2.97
	2023/05	3.80	3.40	3.89	3.80	3.88	3.90	2.70	2.62	2.64
	2023/06	4.20	3.87	3.89	3.80	3.62	3.70	2.70	2.70	2.65
	2023/07	4.90	4.26	4.43	3.70	3.76	3.78	2.70	2.70	2.70
	2023/08	5.30	5.30	5.31	3.80	3.72	3.74	2.70	2.70	2.70
	2023/09	5.30	5.58	5.48	3.70	3.87	3.86	2.70	2.70	2.70
	2023/10	5.30	5.28	5.12	3.90	3.73	3.78	2.90	2.70	2.70
	2023/11	5.20	5.07	5.10	4.00	3.88	3.98	2.90	2.94	2.92
	2023/12	5.10	4.84	4.91	4.10	4.14	4.14	2.90	2.90	2.92
	2024/01	4.20	4.70	4.76	4.30	4.11	4.17	3.20	2.90	2.89
	2024/02	4.20	3.80	4.27	4.40	4.36	4.35	3.40	3.26	3.30
előrejelzés	2024/03		4.16	4.27		4.41	4.35		3.45	3.30
	2024/04		4.19	4.48		4.40	4.36		3.46	3.36
	2024/05		4.28	4.56		4.38	4.33		3.47	3.32
	2024/06		4.23	4.55		4.37	4.25		3.47	3.26
	2024/07		4.20	4.46		4.38	4.16		3.47	3.19
	2024/08		4.24	4.32		4.40	4.05		3.47	3.13

5. melléklet. A tesztadatok, a modellek becslései és előrejelzései.

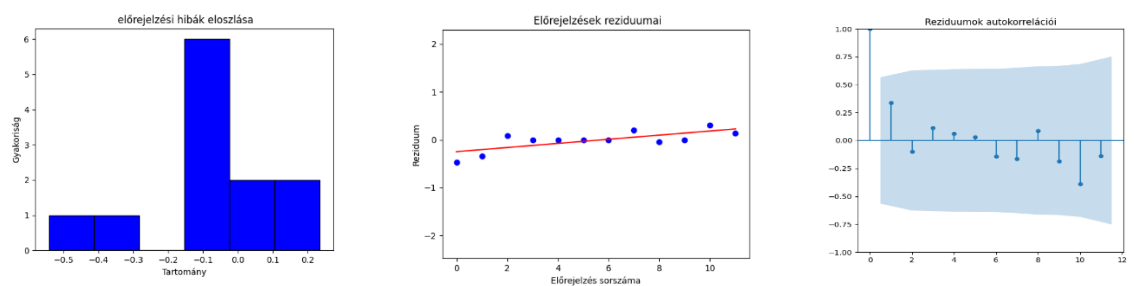
Kovácsna ARIMA (2, 1, 2) teszt hibák



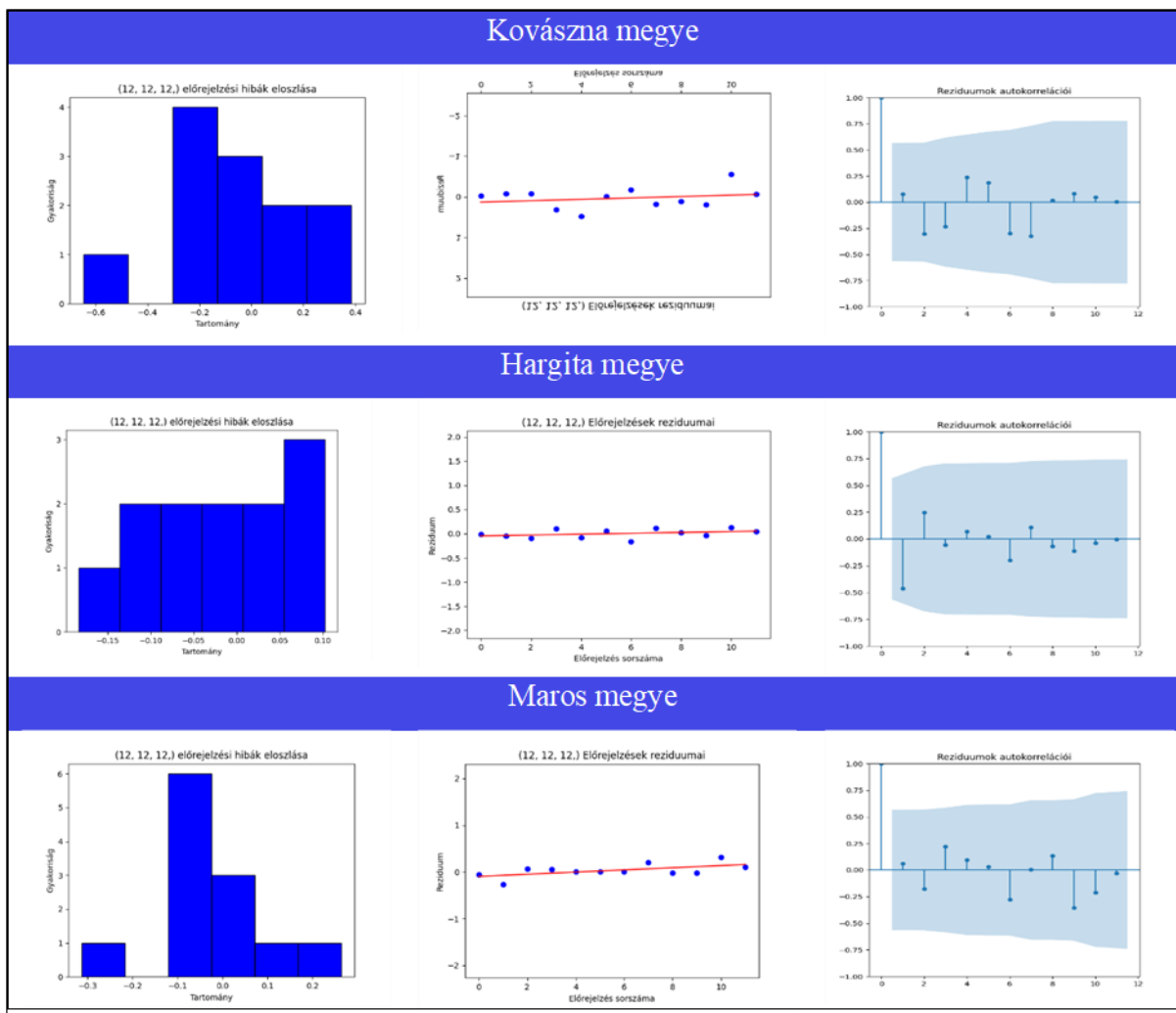
Hargita ARIMA (2, 1, 2) teszt hibák



Maros ARIMA (1, 1, 0) teszt hibák



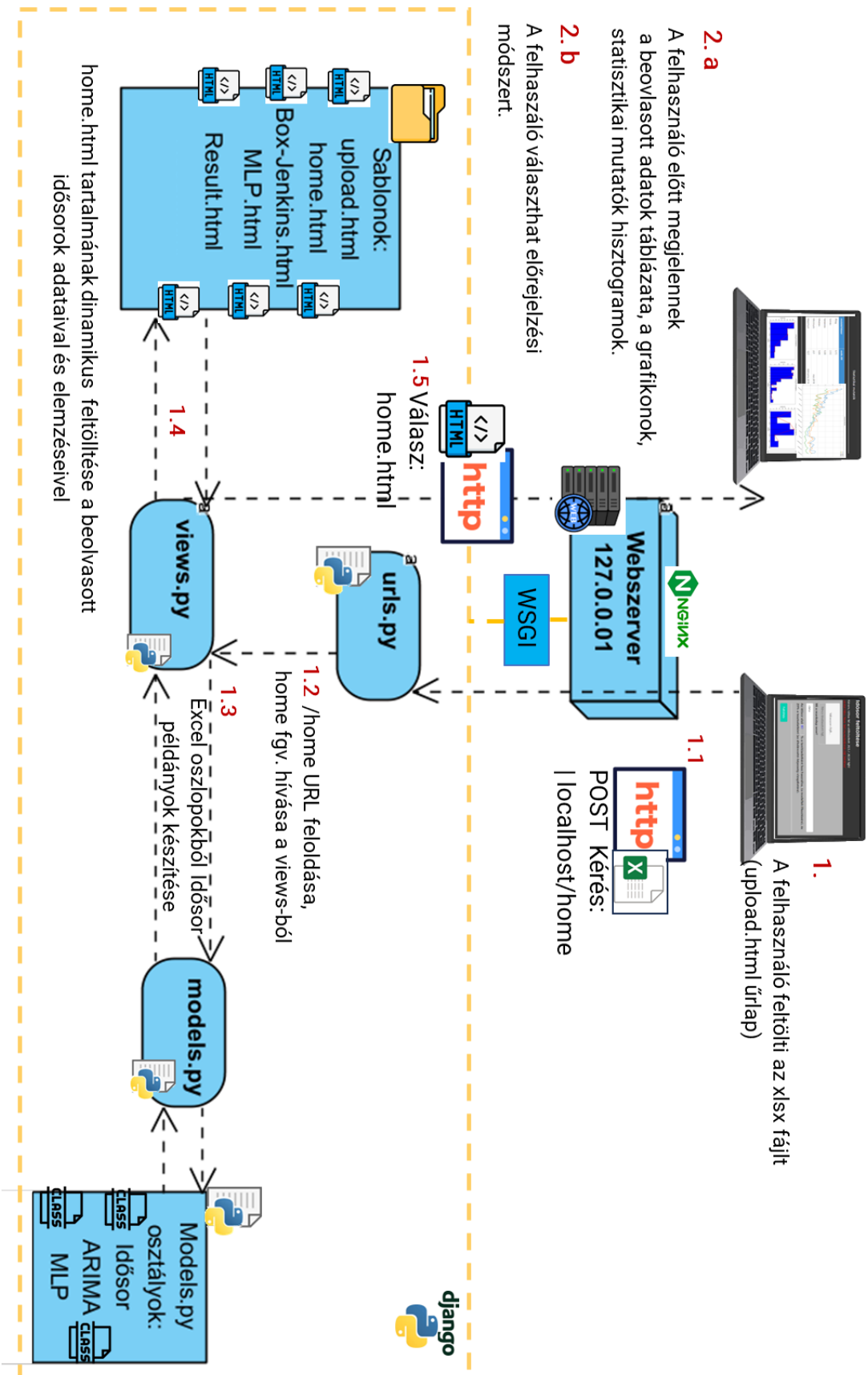
6. melléklet. Az ARIMA modellek teszt hibái.



7. melléklet. Az MLP modellek teszt hibái.

Tanító párok: (amiből megtanulta a súlyokat) 1. [3.33076532 3.79504212 3.88200003] --> 3.769917110242145 2. [3.63443394 3.66916183 3.55172801] --> 3.0050461712390066 3. [3.51296649 3.35446112 2.82512954] --> 2.796445006056333 4. [3.20929787 2.66211956 2.62696633] --> 2.3097089539634257 5. [2.5412269 2.47329913 2.16458549] --> 2.3792426756909837	Teszt párok (amiket meg kell jósoljon): 1. [-0.61692678 -0.54782768 -0.67575395] --> -0.749774802049129 2. [-0.55619306 -0.67370797 -0.74180836] --> -0.8888422455042454 3. [-0.67766051 -0.73664811 -0.87391717] --> -1.0279096889593617 4. [-0.73839423 -0.86252839 -1.00602598] --> -1.0279096889593617 5. [-0.85986168 -0.98840868 -1.00602598] --> -1.0974434106869195
Eltolási értékek vektora: [array([-0.14263795, -0.46405735, 0.37048585, -0.04350344, -0.41589813, 0.0887363, -0.63015977, 0.60406658, -0.33282178, 0.23301269, -0.17569857, 0.12733789]), array([0.34015137, 0.22945146, -0.11284594, -0.06852408, 0.28311273, 0.47685017, 0.36508284, 0.38650221, 0.24352484, -0.41586004, -0.37923313, 0.35262655]),	Rétegek súlyai: [array([-0.09417275, 0.61139253, 0.41652149, 0.08174354, -0.49280698, -0.43675255, -0.51751106, 0.53056216, 0.15568284, 0.1547826, -0.57663247, 0.65966772], [0.51673432, -0.30274285, -0.30353983, -0.4483245, -0.30546888, 0.00814872, -0.04574842, -0.18894728, 0.16956214, -0.53562284, -0.21943787, -0.07219243]), [0.07199837,

8. melléklet. Az MLP modellek tanító adatbázisának, súlyainak és eltolásainak megjelenítése.



9. melléklet. A Django webalkalmazás felépítése.